



TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

**A
MAJOR PROJECT REPORT
ON
OPTIMIZING NEPALI TEXT-TO-SPEECH:
SYSTEMATIC PROGRESSION ACROSS
FASTSPEECH2, VITS, AND SPEECHT5**

SUBMITTED BY
ASHUTOSH BHATTARAI (PUL078BEI007)
APALA TIMALSINA (PUL078BEI008)

SUBMITTED TO
DEPARTMENT OF ELECTRONICS & COMPUTER ENGINEERING

May 8, 2026

Page of Approval

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PULCHOWK CAMPUS

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

The undersigned certifies that they have read and recommended to the Institute of Engineering for progress defense of the project report entitled “**Optimizing Nepali Text-to-Speech: Systematic Progression Across FastSpeech2, VITS, and SpeechT5**” submitted by **Ashutosh Bhattarai, Apala Timalina** in partial fulfillment of the requirements for the Bachelor’s degree in Electronics & Computer Engineering.

.....

Supervisor

Jyoti Tandulkar

Associate Professor

Department of Electronics and Computer Engineering,
Pulchowk Campus, IOE, TU.

Acknowledgments

The successful completion of any project is never the work of a single individual, it is the culmination of guidance, encouragement, and support from many. As we conclude four years of learning and arrive at this final chapter with our final year major project, we consider it our privilege to express our heartfelt gratitude to all those who played a part in making this project come to life.

We would like to begin by expressing our sincere thanks to the Department of Electronics and Computer Engineering, IOE Pulchowk Campus, for providing us the opportunity to work on a meaningful real-world problem as our major project. This experience has not only sharpened our technical and analytical skills, but has also broadened our perspective on what thoughtful engineering can achieve.

We are deeply grateful to our supervisor, **Asst. Prof. Jyoti Tandulkar**, for his invaluable guidance, patience, and insight throughout every stage of this project. His ability to redirect our thinking toward better solutions and his consistent encouragement were crucial in shaping the direction and quality of this work. We also extend our appreciation to our other faculty supervisors and evaluators whose feedback and suggestions helped us refine our approach and strengthen our outcomes.

This project draws on the work of many researchers and contributors in the TTS and speech synthesis community, and we are grateful to all of them. In particular, we acknowledge the team behind the OpenSLR SLR43 dataset (K. Sodimana et al., 2018) [1] and the IndicTTS Nepali Mono Speaker Dataset for making high-quality Nepali speech recordings publicly available. Their contributions to open-source speech resources were foundational to our work.

Finally, and most sincerely, we thank our families and friends for their unwavering support, patience, and belief in us not just through this project, but across these four years.

Authors:

Ashutosh Bhattarai

Apala Timalsina

Abstract

Access to digital written content remains a significant barrier for many Nepali speakers who communicate fluently in the language but face difficulty engaging with written text. Motivated by this accessibility gap, this project attempts the optimization of Text-to-Speech (TTS) systems for Nepali, with the broader goal of improving digital inclusivity through accurate, natural, and computationally efficient speech synthesis.

This work addresses the problem through a systematic, problem-driven exploration of neural TTS for Nepali, a low-resource language with limited speech synthesis infrastructure. Rather than adopting a single-model pipeline, the study documents progressive experimentation across three neural TTS architectures: FastSpeech2 [2], VITS [3], and SpeechT5 [4], each selected for their distinct strengths in improving speech quality, naturalness, and training efficiency under constrained-data conditions.

The project involved building a Nepali-specific data processing pipeline from the ground up, including grapheme-to-phoneme (G2P) and grapheme-to-grapheme (G2G) lexicon constructions, along with phoneme dictionary generation and phoneme set definition using eSpeak NG, configuration and testing of the Montreal Forced Aligner (MFA) [5], and an extensive text normalization pipeline to handle the linguistic characteristics of the Nepali script. Multiple training configurations, hyperparameter tuning strategies, and dataset variations were systematically explored across all three models to identify effective practices for TTS development under limited data conditions. Given the scarcity of prior Nepali-focused experimentation on tools such as MFA and G2P phonemizers, methodologies were adapted from related low-resource language studies, including work on Hindi, Sanskrit, and other Indic languages.

FastSpeech2 [2] served as the initial point of experimentation, implemented from scratch for its explicit prosodic control over pitch, energy, and duration, as well as its efficient non-autoregressive parallel inference. Experimentation then extended to VITS and SpeechT5, both pre-trained on the LJSpeech dataset [6] and subsequently fine-tuned on the same Nepali dataset, enabling a controlled and fair comparison across all three systems. The overall pipeline additionally incorporates Optical Character Recognition (OCR) to accept Nepali text from real-world digital sources including PDFs, screenshots, and textual images before passing it into the speech synthesis pipeline.

Speech quality was evaluated using Mean Opinion Score (MOS), and the findings provide useful insight into the challenges of low-resource TTS development, the trade-offs between native training and transfer learning and the critical role of alignment quality and data sufficiency in producing intelligible and expressive Nepali speech. This work establishes both a functional end-to-end pipeline and a strong experimental baseline for future implementation of higher quality Nepali TTS systems.

Keywords: *Nepali Text-to-Speech, FastSpeech2, VITS, SpeechT5, Mel-spectrogram, HiFi-GAN, Speech Synthesis, Low-Resource Language*

Contents

Acknowledgements	iii
Abstract	iv
Contents	vii
List of Figures	viii
List of Tables	ix
List of Abbreviations	x
1 Introduction	1
1.1 Background	1
1.2 Problem Statement	1
1.3 Objectives	2
1.4 Scope	2
2 Literature Review	3
2.1 Related Works	3
2.1.1 Nepali Text-to-Speech Systems	3
2.1.2 Challenges in Nepali TTS Pipeline Development	4
2.1.3 Neural TTS Architectures	4
2.2 Related theory	5
2.3 Overview of TTS Evolution	5
2.4 Implementation of TTS Systems	6
2.4.1 Text Input and Extraction	6
2.5 Text Normalization and Preprocessing	6
2.5.1 Grapheme-to-Phoneme and Grapheme-to-Grapheme Conversion	7
2.5.2 Audio Preprocessing	8
2.6 Mel-Spectrogram	8
2.6.1 Role in TTS Systems	8
2.7 FastSpeech2	8
2.7.1 Architecture	9
2.7.2 Montreal Forced Aligner (MFA)	9
2.7.3 Nepali-Specific Configuration for FastSpeech2	10
2.8 VITS	10
2.8.1 Architecture	10
2.8.2 Training Objective	11
2.8.3 Nepali-Specific Configuration for VITS	11

2.9	SpeechT5	12
2.9.1	Architecture	12
2.9.2	Nepali-Specific Configuration for SpeechT5	12
2.10	Vocoder: Mel-Spectrogram to Waveform Synthesis	13
2.10.1	HiFi-GAN	14
2.11	Evaluation Metrics for the TTS System	14
2.12	Nepali Language and Devanagari Script	15
2.12.1	Structure of the Devanagari Script	15
2.12.2	Unicode Encoding of Devanagari	16
2.12.3	Conjunct Consonants and the Halanta	17
2.12.4	The Schwa and Schwa Deletion in Nepali	17
2.12.5	Text Normalization for Nepali	18
2.12.6	Tokenization and Syllabification	19
2.12.7	Transliteration	19
3	Methodology	20
3.1	Feasibility Study	20
3.2	Requirement Analysis	22
3.3	Methodology Pipeline	23
3.3.1	Data Collection	26
3.3.2	Data Preprocessing	27
3.3.3	FastSpeech2 : Forced Alignment and Feature Preparation with Preprocessing	29
3.3.4	VITS : Direct Character Embedding and Preprocessing	31
3.3.5	SpeechT5 : Transliteration and Preprocessing	31
3.3.6	Training Iteration and Refinement	32
3.3.7	Testing and Evaluation	33
4	Experimentation & Results	34
4.1	FastSpeech2 Model Training - Cycle 1 (SLR43 Dataset)	34
4.1.1	Acoustic and Model Configuration	34
4.1.2	Training Setup	35
4.2	FastSpeech2 Model Training - Cycle 2 (SLR43 Dataset)	36
4.3	FastSpeech2 Model Training - Cycle 3 (IndicTTS Dataset)	38
4.4	FastSpeech2 Model Finetuning - Cycle 4 (IndicTTS Dataset)	39
4.5	FastSpeech2 Model Finetuning - Cycle 5 (Finetuning on LJSpeech Model with SLR43 Dataset)	40
4.6	VITS Fine-Tuning on an LJSpeech Checkpoint with the IndicTTS Dataset	41
4.6.1	Fine-tuning Setup and Configuration	41
4.6.2	Results and Audio Quality	43
4.7	SpeechT5 Finetuning with SLR43	43
4.7.1	Fine-tuning Setup and Configuration	43
4.7.2	Results and Audio Quality	45

5	Results and Comparative Evaluation	46
5.1	Discussion	47
5.1.1	FastSpeech2	47
5.1.2	VITS	48
5.1.3	SpeechT5	48
6	Conclusion	50
7	Limitations and Future Enhancements	51
7.1	Limitations	51
7.2	Future Enhancements	52
	References	53

List of Figures

2.1	TTS Architecture	6
2.2	Structure of a Nepali Word	7
2.3	FastSpeech2 Architecture	9
2.4	VITS Architecture	11
2.5	SpeechT5 Architecture	12
2.6	TTS system pipeline comparison across FastSpeech2, VITS, and SpeechT5	13
2.7	Nepali transliteration example used in the preprocessing pipeline	19
3.1	Methodology of the Fastspeech2 - Nepali TTS pipeline from scratch.	23
3.2	Methodology of the VITS - Nepali TTS pipeline using Transfer Learning.	24
3.3	Methodology of the SpeechT5 - Nepali TTS pipeline using Transfer Learning.	25
3.4	Montreal Forced Aligner	29
3.5	Train Val split representation for preprocessing	31
4.1	MelSpectrogram for Sample Text	35
4.2	Audio Waveform of Generated Audio	35
4.3	Figure: Loss Graphs for run 1	36
4.4	Figure: Loss Graph for the phoneme based run	37
4.5	Figure: Loss Graph obtained in run 3 changing parameters	38
4.6	Figure: Loss Graph obtained in run 4 changing parameters	39
4.7	Figure: Loss Graph obtained in run 5 with Transfer Learning	40
4.8	Figure: Loss Graph for VITS over 50k steps of training	42
4.9	Figure: Loss Graph for SpeechT5 over 5000 steps of training	45

List of Tables

2.1	Summary of Neural Vocoders	14
2.2	Other Evaluation Metrics for Nepali TTS System	15
2.3	Key Unicode Code Points for Nepali Devanagari	16
3.1	Summary of Nepali Speech Datasets Used	27
3.2	Comparison of Grapheme-Based and Cluster-Based Representation in Nepali	28
3.3	Input File Structure for Forced Alignment	29
3.4	Acoustic Features Extracted for FastSpeech2 Training	30
3.5	Representation of Nepali in its corresponding English transliteration	32
4.1	Sample Nepali Sentences and Corresponding Grapheme Decomposition	34
4.2	FastSpeech2 Model Configuration	34
4.3	Training Hyperparameters	35
4.4	FastSpeech2 Loss Values of Cycle-1	35
4.5	Sample Nepali Sentences and Phoneme Representations	36
4.6	FastSpeech2 Loss Metrics of Cycle-2	36
4.7	FastSpeech2 Training – Cycle 3 (IndicTTS Dataset)	38
4.8	FastSpeech2 Loss Metrics of Training Cycle-4	39
4.9	FastSpeech2 Loss Metrics of Training Cycle-5	40
4.10	FastSpeech2 Training Parameters (Optimizer and Step Settings)	40
4.11	VITS Fine-tuning Training Parameters	41
4.12	SpeechT5 fine-tuning hyperparameters on SLR43	44
5.1	Comparative Overview of FastSpeech2, VITS, and SpeechT5	46
5.2	Comparison of Mean Opinion Scores (MOS) Across Prior Nepali TTS Systems and the Proposed Approaches	47

List of Abbreviations

AI	Artificial Intelligence
ASR	Automatic Speech Recognition
CNN	Convolutional Neural Network
DNN	Deep Neural Network
DSP	Digital Signal Processing
GAN	Generative Adversarial Network
G2P	Grapheme-to-Phoneme
GPU	Graphics Processing Unit
JSON	JavaScript Object Notation
LSTM	Long Short-Term Memory
ML	Machine Learning
MFCC	Mel Frequency Cepstral Coefficients
Mel	Mel-Spectrogram
NFC	Nepali Font Converter
NLP	Natural Language Processing
OCR	Optical Character Recognition
OpenSLR	Open Speech and Language Resources
ReLU	Rectified Linear Unit
RNN	Recurrent Neural Network
SLR	Speech and Language Resources
TTS	Text-to-Speech
WAV	Waveform Audio File Format
WER	Word Error Rate
SR	Sampling Rate
HiFi-GAN	High-Fidelity Generative Adversarial Network
MOS	Mean Opinion Score

1. Introduction

1.1 Background

Text-to-Speech (TTS) synthesis is the task of converting written text into natural-sounding spoken audio. Early TTS systems relied on concatenative and formant-based synthesis, producing intelligible but unnatural output through rule-based methods. The introduction of deep learning transformed the field, with end-to-end neural architectures dramatically improving speech naturalness and expressiveness.

Modern neural TTS pipelines typically consist of two components: an acoustic model that converts text into an intermediate representation such as a mel-spectrogram, and a vocoder that converts that representation into a time-domain waveform. Architectures such as FastSpeech2 [2], VITS [3], and SpeechT5 [4] represent prominent points along this evolution, each offering distinct approaches to the trade-offs between speech quality, inference speed, and model complexity.

Despite significant progress, the majority of advances in neural TTS have been concentrated in high-resource languages where large speech corpora and mature linguistic tooling are readily available. Extending these systems to low-resource languages such as Nepali introduces foundational challenges that go beyond model selection alone, requiring pipeline construction, lexicon development, and alignment configuration that high-resource settings rarely demand.

1.2 Problem Statement

Modern Text-to-Speech systems have achieved remarkable levels of naturalness and intelligibility, largely driven by the availability of large, high-quality speech corpora and the computational resources required to train deep neural architectures. Languages such as English, Mandarin, and Hindi have benefited enormously from this progress, with production-grade TTS systems now widely accessible through both commercial platforms and open-source frameworks. These systems typically rely on hundreds of hours of clean, studio-recorded speech data, well-established grapheme-to-phoneme dictionaries, and mature linguistic tooling that has been refined over decades of research and development.

Nepali, despite being spoken by over 17 million native speakers across Nepal and parts of India, occupies a very different position in this landscape. The language remains critically underserved in speech technology, with virtually no widely adopted native TTS system built specifically for it. The challenges are structural. Nepali lacks the large annotated speech corpora, standardized pronunciation lexicons, and language-specific acoustic tooling. Its Devanagari script introduces additional complexity, as phoneme boundaries, schwa deletion patterns, and script-to-sound correspondences do not map trivially onto pipelines designed for Latin-script languages. Consequently, approaches that work seamlessly for English or Hindi often require significant adaptation, approximation, or rebuilding from scratch when applied to Nepali.

This gap has practical consequences beyond research. A wide range of potential applications from assistive tools for individuals with visual impairments and limited literacy, to automated reading of government notices, news articles, and educational content, all remain unrealized simply because accessible, high-quality Nepali TTS does not exist at scale. Notably, several mainstream Nepali news portals currently resort to manually recorded audio clips for their read-aloud features, a workaround that is neither scalable nor cost-effective. A reliable, automated Nepali TTS system would eliminate this dependency and open the door

to a broad spectrum of accessibility and content delivery applications across education, journalism, public services, and assistive technology.

It is within this context that the present project situates itself not as a single-model deployment, but as a systematic investigation into what it takes to build, configure, and optimize neural TTS for a low-resource language like Nepali, and what the current state of available architectures can and cannot achieve under such constraints.

1.3 Objectives

- To implement FastSpeech2 from scratch for Nepali speech synthesis, experimenting across multiple training configurations, hyperparameter settings, and alignment strategies to maximize synthesis quality under limited data conditions.
- To fine-tune VITS and SpeechT5 on Nepali speech data, leveraging transfer learning to evaluate each model’s strengths and how pre-trained models can outperform a natively trained system, under the same low-resource constraints.
- To comparatively evaluate FastSpeech2, VITS, and SpeechT5 in terms of speech naturalness and intelligibility through Mean Opinion Score (MOS) drawing insights into the strengths and limitations of each approach for Nepali TTS.
- To develop an end-to-end Nepali TTS application with integrated OCR, enabling speech synthesis directly from real-world digital sources such as PDFs, screenshots, and textual images.

1.4 Scope

The scope of this project is defined as follows:

- The system focuses on standard digital Nepali text input. It does not support mixed-language content, colloquial speech patterns, or handwritten text.
- The system generates audio output from digital text and does not perform real-time speech recognition or translation.
- As a planned extension, the project aims to incorporate the best performing TTS pipeline into a publicly accessible web extension, with the possibility of further integrating a speech-to-text component to create a complete bidirectional speech interface.

This project lays the foundation for future expansion by enabling support for additional low-resource languages and integrating advanced neural TTS models for further enhancement of speech synthesis.

2. Literature Review

2.1 Related Works

2.1.1 Nepali Text-to-Speech Systems

Several research efforts have contributed to the development of Nepali Text-to-Speech (TTS) systems, progressing from early concatenative approaches to modern neural architectures. **Bajracharya et al. (2019)** [7] developed one of the earliest Nepali TTS systems using the Festival Speech Synthesis framework, employing a concatenative synthesis method based on unit selection to generate natural-sounding Nepali speech. While the system demonstrated intelligible output and was integrated with the NVDA screen reader, it suffered from slow synthesis response times, phoneme misalignment artifacts, and limited expressiveness fundamental constraints of concatenative approaches that motivated the shift toward neural synthesis methods. **Basnet (2021)** [8] introduced the autoregressive WaveNet model for Nepali speech synthesis, trained on the SLR43 and SLR54 datasets. This work achieved better intelligibility and fluency than concatenative systems but faced limitations in prosody and accent due to speaker variability and data quality constraints. The system achieved a Mean Opinion Score (MOS) of 3.07 generally considered acceptable but indicative of remaining room for improvement in naturalness and expressiveness.

Khadka et al. (2023) [9] further demonstrated that attention-based autoregressive models could produce high-quality Nepali speech, implementing a Tacotron 2 acoustic model paired with a HiFi-GAN vocoder for Nepali mel-spectrogram generation. The system achieved a Mean Opinion Score (MOS) of 4.04, establishing a strong naturalness benchmark for Nepali TTS. However, autoregressive models like Tacotron 2 remain inherently slow at inference due to their sequential generation process, motivating exploration of non-autoregressive alternatives.

Ren et al. (2020) [10] addressed this bottleneck with FastSpeech, a non-autoregressive transformer-based model that bypasses sequential dependencies through parallel mel-spectrogram generation. Overcoming the limitations of student-teacher distillation pipelines, FastSpeech achieved mel-spectrogram generation up to 270 times faster and end-to-end speech synthesis up to 38 times faster than autoregressive transformer baselines making it particularly compelling for low-resource and deployment-constrained settings. **Lamichhane et al. (2023)** [11] presented one of the earliest applications of this TTS model for Nepali, implementing FastSpeech with a MelGAN vocoder. Their system achieved faster inference compared to autoregressive approaches, though challenges in prosody modeling and dataset limitations persisted. Notably, this work did not employ forced alignment tools such as the Montreal Forced Aligner (MFA), which is critical for duration prediction in non-autoregressive models a gap this project directly addresses.

Kumar et al. (2023) [12] conducted a comparative evaluation of neural TTS architectures across thirteen Indian languages, concluding that pairing FastPitch with HiFi-GAN produced the most natural and efficient synthesized speech among the tested configurations with the MOS score of upto 4.00 for Hindi languages. A subsequent **Nepali TTS study (2023)** [13] implemented the same FastPitch HiFi-GAN combination on the OpenSLR and ILPRL-NAB datasets, achieving MOS scores of 3.70 and 3.40 respectively.

While these results marked progress, neither of those studies extended experimentation to more recent architectures such as FastSpeech2, VITS, or SpeechT5, leaving a clear gap in the literature regarding how newer models perform under Nepali low-resource conditions.

2.1.2 Challenges in Nepali TTS Pipeline Development

A recurring and underaddressed challenge across Nepali TTS research is the absence of mature linguistic tooling for the language. The Montreal Forced Aligner (MFA) [5], widely adopted for forced alignment in English and other high-resource languages, does not natively support Nepali Devanagari script. MFA requires a word-to-phoneme pronunciation dictionary and a compatible acoustic model as prerequisites neither of which exists in a standardized form for Nepali. Constructing these resources from scratch demands significant effort in phoneme set definition, lexicon development, and iterative alignment testing, yet this process is rarely documented in existing Nepali TTS literature.

Similarly, grapheme-to-phoneme (G2P) conversion for Nepali remains an open problem. Most existing G2P tools are designed for Latin-script languages and do not generalize to Devanagari script. A notable workaround adopted widely across Nepali and other Indic-language TTS systems is the use of transliteration converting Devanagari script into a Latin-based representation prior to phonemization rather than performing true grapheme-to-phoneme or grapheme-to-grapheme conversion directly on the native script. While this approach sidesteps the absence of Devanagari-native phonemizers, it introduces an additional processing step and risks losing script-specific phonetic nuances that are not preserved through transliteration. This reliance on transliteration reflects the broader absence of robust, Devanagari-native G2P tooling for Nepali rather than a deliberate architectural choice.

Ghimire et al. [14] addressed aspects of this problem in their work on pronunciation-aware syllable tokenization for Nepali Automatic Speech Recognition, providing a useful reference for phoneme boundary definition and syllabic structure in Nepali. However, a robust, reusable G2P pipeline operating directly on Nepali script for TTS applications remains absent from the literature.

This absence of foundational tooling means that virtually every Nepali TTS research effort that does not depend on transliteration must rebuild core pipeline components, such as grapheme-to-phoneme lexicons, independently, with limited ability to build directly upon prior work. This project directly confronts this challenge through systematic lexicon construction, MFA configuration and testing across multiple alignment strategies, and a text normalization pipeline developed to handle Nepali script natively without reliance on transliteration.

2.1.3 Neural TTS Architectures

The **FastSpeech2** [2] TTS model is a non-autoregressive acoustic model that predicts mel-spectrograms in parallel, bypassing the sequential dependencies of autoregressive models. It explicitly models prosodic variance through dedicated duration, pitch, and energy predictors, addressing the one-to-many mapping problem inherent in TTS. The original paper reported inference speeds up to 38 times faster than autoregressive baselines on English and Mandarin, with competitive speech quality. However, no prior literature documents the application of FastSpeech2 to Nepali, nor its combined use with MFA-based forced alignment for a Devanagari-script language making this work among the first systematic explorations of this configuration.

Variational Inference with adversarial learning for end-to-end Text-to-Speech - **VITS** [3] is an end-to-end generative model that unifies acoustic modelling and waveform synthesis within a single variational autoencoder framework, eliminating the need for a separate vocoder. VITS employs normalizing flows and

adversarial training to produce highly natural speech, achieving strong results on English benchmarks. Its end-to-end nature and relatively compact training requirements make it appealing for low-resource settings, though its performance under limited Nepali data conditions has not been previously studied.

SpeechT5 [4] is a unified encoder-decoder framework pre-trained on large amounts of unlabelled speech and text data, designed to handle multiple speech processing tasks including TTS, ASR, and voice conversion within a single model. Its transformer-based architecture and cross-modal pre-training allow it to leverage rich linguistic knowledge for improved speech synthesis, making it well-suited for fine-tuning on low-resource languages. Despite its versatility and strong performance on English benchmarks, experimentation with SpeechT5 on Nepali data remains absent from the existing literature.

The absence of documented experimentation with FastSpeech2, VITS, and SpeechT5 on Nepali combined with the lack of MFA-based alignment pipelines for the language forms the primary research gap this project addresses.

2.2 Related theory

2.3 Overview of TTS Evolution

The field of Text-to-Speech (TTS) synthesis has evolved remarkably over the past two centuries. Early attempts involved mechanical speaking machines that imitated human vocal organs. Significant progress began in the mid-20th century, when Noriko Umeda and colleagues developed one of the first fully functional TTS systems for English. While intelligible, the speech lacked fluency and naturalness.

Subsequent developments including concatenative synthesis, which joined pre-recorded speech segments, and parametric synthesis using Hidden Markov Models (HMMs) improved the smoothness and controllability of generated speech. Deep learning ushered in a transformative era: WaveNet (2016) [15] produced high-fidelity waveform-level audio through autoregressive modelling; Tacotron [16] and Tacotron 2 [17] combined sequence-to-sequence architectures with attention mechanisms to significantly improve prosody and expressiveness; and FastSpeech [10] and FastSpeech2 [2] introduced non-autoregressive architectures for faster, more stable synthesis without compromising naturalness.

A significant architectural leap came with **VITS** [3], which eliminated the traditional two-stage TTS pipeline entirely by unifying acoustic modelling and waveform synthesis within a single end-to-end variational framework. By combining normalizing flows, variational inference, and adversarial training, VITS produced remarkably natural speech while also removing the dependency on external alignment tools through its internal monotonic alignment search mechanism. Around the same time, transformer-based pre-training approaches began influencing TTS. **SpeechT5** [4] extended the T5 encoder-decoder framework to speech and language tasks, enabling a single pre-trained model to perform TTS, ASR, and voice conversion — demonstrating that large-scale cross-modal pre-training could benefit speech synthesis, particularly for low-resource languages where labelled data is scarce.

The pace of advancement has accelerated further in recent years. Models such as **NaturalSpeech2** [18], **StyleTTS2** [19], and **Voicebox** [20] have pushed the boundaries of speech naturalness, style control, and zero-shot voice cloning through diffusion-based and flow-matching generative approaches. Large language model-driven TTS systems such as **VALL-E** [21] have further demonstrated that treating speech synthesis as a language modelling problem -predicting discrete audio tokens from text, can achieve near-human naturalness with only a few seconds of reference audio. These developments signal a broader shift in the field toward generative, data-scalable approaches that increasingly blur the boundary between speech synthesis and speech generation.

Despite this rapid progress, the majority of advances remain concentrated in high-resource languages. For low-resource languages such as Nepali, the absence of large speech corpora, standardized linguistic tooling, and language-specific model support means that even well-established architectures like FastSpeech2 and VITS require significant adaptation effort. The present project situates itself within this gap, systematically implementing and evaluating three prominent neural TTS architectures for Nepali, contributing both a functional pipeline and experimental findings to an underserved area of speech synthesis research.

2.4 Implementation of TTS Systems

A modern neural Text-to-Speech system typically consists of a sequential pipeline of components, each responsible for a distinct stage of processing starting from raw text input to final audio output. While the exact architecture varies across models, most pipelines share common stages: text normalization and preprocessing, phoneme or token conversion, acoustic modeling, and waveform synthesis. Three TTS architectures were implemented and evaluated in this project : FastSpeech2, VITS, and SpeechT5, each following a distinct internal pipeline. FastSpeech2 follows the classical two-stage approach of explicit mel-spectrogram prediction followed by vocoding. VITS departs from this by unifying acoustic modeling and waveform synthesis within a single end to end framework, bypassing the explicit mel-spectrogram entirely. SpeechT5 adopts a pre-trained encoder-decoder paradigm, leveraging cross-modal pre-training before fine-tuning on Nepali data. Each model required distinct preprocessing strategies, alignment configurations, and training pipelines tailored to the Nepali language. Figure 2.1 illustrates the general architecture of a neural TTS pipeline.

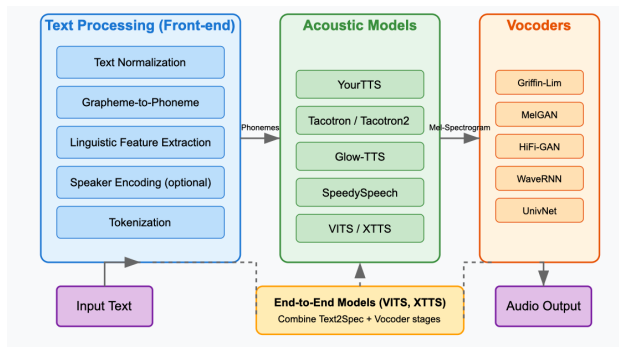


Figure 2.1: TTS Architecture

2.4.1 Text Input and Extraction

The system accepts Nepali text from multiple digital input sources, including plain text, digital documents, text screenshots, and images containing Nepali text. For inputs that are not directly machine-readable, such as images and scanned documents, Optical Character Recognition (OCR) is employed to extract the underlying textual content before it is passed further down the pipeline.

2.5 Text Normalization and Preprocessing

After text extraction, normalization ensures consistency and compatibility across all three TTS pipelines. Raw Nepali text whether extracted via OCR or provided directly, may contain numerals, punctuation irregularities, abbreviations, and script inconsistencies that must be resolved before acoustic modeling. Key normalization steps include:

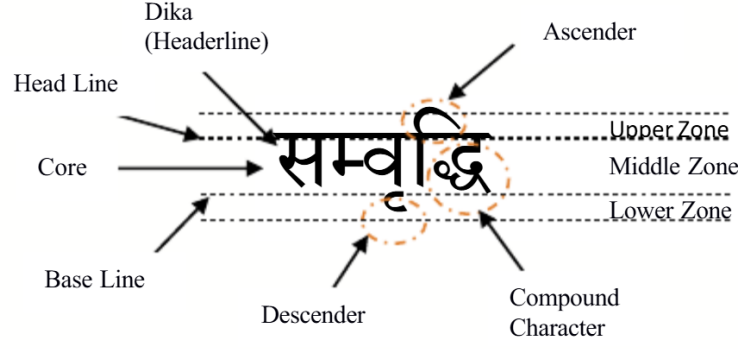


Figure 2.2: Structure of a Nepali Word

- **Number Expansion:** Devanagari numerals (०-९) and their Arabic equivalents are converted to their full Nepali word equivalents to ensure correct pronunciation.
- **Punctuation Handling:** Inconsistent or ambiguous punctuation marks are standardized or removed to prevent misinterpretation during phoneme conversion.
- **Abbreviation Expansion:** Commonly used short forms and abbreviations are expanded to their full spoken equivalents.
- **Script Standardization:** Unicode normalization is applied to resolve inconsistencies in Devanagari character encoding, particularly for conjunct characters and vowel modifiers.

2.5.1 Grapheme-to-Phoneme and Grapheme-to-Grapheme Conversion

The Grapheme-to-Phoneme (G2P) conversion is a critical intermediary stage in TTS pipelines, transforming written characters into phonemic representations that correspond to how words are pronounced. In Nepali, which uses the Devanagari script, G2P mapping is particularly complex due to inherent vowel rules, schwa deletion patterns, and pronunciation variations based on word position and context. Two primary approaches were explored in this project:

- **Grapheme-to-Grapheme (G2G):** Each Devanagari character is directly tokenized and used as-is, without explicit phoneme mapping. This approach is computationally simple and avoids the need for a pronunciation lexicon, but may overlook contextual phonetic nuances specific to Nepali.
- **Grapheme-to-Phoneme (G2P) / Akshara-based:** Text is segmented into aksharas — visual syllabic units consisting of consonant clusters, vowels, and modifiers — and mapped to their phonemic equivalents. This approach aligns more closely with the phonetic structure of Nepali speech and produces more accurate duration and prosody predictions in non-autoregressive models.

A key challenge encountered in this work was the absence of a standardized, Devanagari-native G2P tool for Nepali. Many existing TTS systems for Nepali and related Indic languages rely on transliteration converting Devanagari to a Latin-based representation prior to phonemization as a workaround for this gap. While functional, transliteration introduces an additional processing step and risks losing script-specific phonetic nuances. In this project, a custom pronunciation lexicon was constructed directly from Nepali Devanagari script to avoid this dependency, informed by phoneme boundary definitions from [14].

2.5.2 Audio Preprocessing

Alongside text preprocessing, audio data requires its own preparation pipeline before being used for acoustic model training. Key steps include:

- **Resampling:** All audio files were resampled to a consistent sampling rate of 22,050 Hz to ensure compatibility across all three TTS pipelines.
- **Format Standardization:** Audio files were converted to mono-channel 16-bit PCM WAV format to meet the input requirements of MFA and all three acoustic models.
- **Silence Trimming:** Leading and trailing silences were removed from audio clips to reduce alignment errors and improve training stability.
- **Mel-Spectrogram Extraction:** For FastSpeech2, mel-spectrograms were pre-extracted from audio using consistent STFT parameters : including FFT size, hop length, and mel filterbank configuration, to serve as training targets for the acoustic model.

2.6 Mel-Spectrogram

A Mel-Spectrogram is a time–frequency representation of an audio signal, commonly used as an intermediate feature in modern TTS systems. It captures how the frequency content of a signal changes over time, mapped onto the Mel scale (an approximation of the non-linear way humans perceive pitch). Since human auditory perception is more sensitive to differences in lower frequencies than higher ones, the Mel scale spaces frequencies in a perceptually uniform manner, making it well-suited for speech modeling.

In TTS pipelines, the raw waveform is first transformed into a spectrogram using the Short-Time Fourier Transform (STFT), dividing the signal into short overlapping frames and computing the frequency content for each. The frequency axis is then converted to the Mel scale, producing the final Mel-Spectrogram that captures energy distribution across perceptually relevant frequency bands over time.

2.6.1 Role in TTS Systems

In TTS pipelines, the Mel-Spectrogram serves as the key intermediate representation between text and the final audio waveform. Neural TTS models including FastSpeech2, Tacotron 2, and Transformer TTS, first predict the Mel-Spectrogram from input text or phonemes, encoding essential prosodic features such as intonation, rhythm, and stress. The predicted Mel-Spectrogram is then passed to a vocoder for waveform reconstruction. This two-stage pipeline (text-to-Mel and Mel-to-waveform) forms the basis of most modern neural TTS systems. VITS departs from this paradigm by unifying both stages within a single end-to-end model, eliminating the explicit Mel-Spectrogram as an intermediate representation.

For Nepali TTS specifically, the Mel-Spectrogram helps capture the rich vowel sounds, tonal variations, and syllabic timing characteristic of Nepali speech, ensuring that synthesized audio retains naturalness and intelligibility.

2.7 FastSpeech2

FastSpeech2 [2] is a non-autoregressive TTS model that predicts mel-spectrograms in parallel, overcoming the slow sequential inference and attention instability issues found in autoregressive models such as Tacotron 2. It explicitly models prosodic variance through a dedicated variance adaptor, addressing the one-to-many mapping problem inherent in TTS where the same text can be spoken in multiple valid ways differing in pitch, speed, and energy.

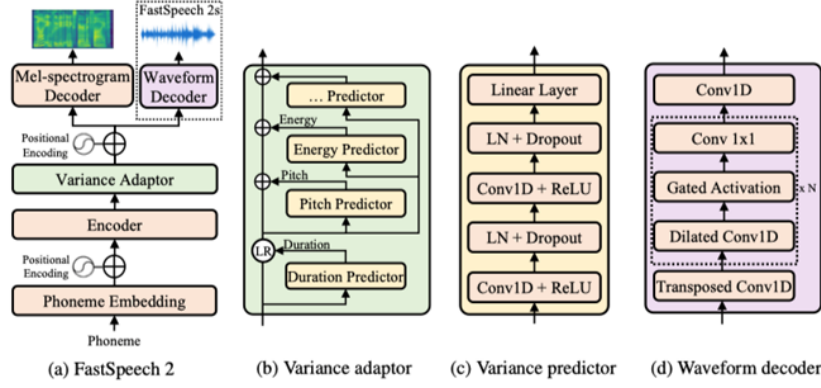


Figure 2.3: FastSpeech2 Architecture

2.7.1 Architecture

- **Encoder:** A Feed-Forward Transformer (FFT) encoder processes phoneme embeddings and converts them into contextual hidden representations, capturing dependencies across the input phoneme sequence.
- **Variance Adaptor:** Predicts three prosodic features from the encoder output duration, pitch (F0), and energy each through a dedicated lightweight predictor network. Duration predictions are used to regulate the length of each phoneme in the output, aligning the encoder output with the mel-spectrogram length.
- **Decoder:** A second FFT-based decoder takes the length-regulated encoder output and generates the final mel-spectrogram frames in parallel.

FastSpeech2 is trained using mel-reconstruction loss (L1) alongside auxiliary variance prediction losses for duration, pitch, and energy. This design enables fast, stable training and inference without sacrificing speech naturalness, making it well-suited for low-resource languages like Nepali.

2.7.2 Montreal Forced Aligner (MFA)

Forced alignment is a prerequisite for FastSpeech2 training, as the model requires precise phoneme-level duration targets extracted from real speech. The Montreal Forced Aligner (MFA) [5] is an open-source tool developed at McGill University that aligns speech audio with its corresponding text transcription at the phoneme and word level, producing precise time boundaries for each phoneme in the dataset.

MFA does not natively support Nepali Devanagari script and requires two primary inputs:

1. **Pronunciation Dictionary:** A word-to-phoneme mapping for every token in the dataset. For Nepali, this dictionary was constructed from scratch, as no standardized Nepali pronunciation lexicon exists. Both grapheme-based (G2G) and phoneme-based (G2P) dictionaries were constructed and tested across multiple alignment experiments.
2. **Paired Audio and Transcription Files:** WAV audio files with corresponding .LAB transcription files, where every token in the transcription must exist in the pronunciation dictionary. Any out-of-vocabulary tokens cause alignment failures, requiring iterative dictionary refinement.

Since no pre-existing MFA acoustic model was available for Nepali, an acoustic model was trained from scratch using the Nepali dataset prior to alignment. The output of MFA is a **TextGrid** file for each audio-transcription pair, containing aligned time boundaries at both the word and phoneme level, which are subsequently used to extract duration targets for FastSpeech2 training.

2.7.3 Nepali-Specific Configuration for FastSpeech2

Adapting FastSpeech2 for Nepali required several language-specific configurations beyond standard English setups:

- Definition of a custom Nepali phoneme set compatible with both the pronunciation dictionary and MFA alignment output.
- Configuration of pitch and energy extraction parameters suited to the acoustic range of the Nepali speaker in the dataset.
- Multiple training runs with varying hyperparameters including learning rate, batch size, and training steps to identify stable convergence behaviour under limited data conditions.
- Iterative refinement of the pronunciation dictionary based on MFA alignment failure logs to maximize successfully aligned training samples.

2.8 VITS

VITS [3] (Variational Inference with adversarial learning for end-to-end Text-to-Speech) is an end-to-end generative TTS model that unifies acoustic modelling and waveform synthesis within a single framework, eliminating the need for a separate vocoder. Unlike FastSpeech2, VITS does not require forced alignment as a preprocessing step instead learning its own internal alignment during training through a monotonic alignment search (MAS) mechanism.

2.8.1 Architecture

- **Posterior Encoder:** Takes mel-spectrograms as input during training and encodes them into a latent representation using non-causal WaveNet residual blocks.
- **Prior Encoder (Text Encoder):** A transformer-based encoder processes input phonemes and produces a prior distribution over the latent space, conditioned on the input text.
- **Normalizing Flow:** A series of affine coupling layers that transform the prior distribution to better match the posterior, improving the expressiveness of the latent representation.
- **Stochastic Duration Predictor:** Learns phoneme duration distributions rather than fixed durations, enabling natural variation in speech rhythm without requiring external alignment tools.
- **Decoder (HiFi-GAN Generator):** Directly synthesizes waveforms from the latent representation using a HiFi-GAN-based generator, bypassing the explicit mel-spectrogram stage entirely.

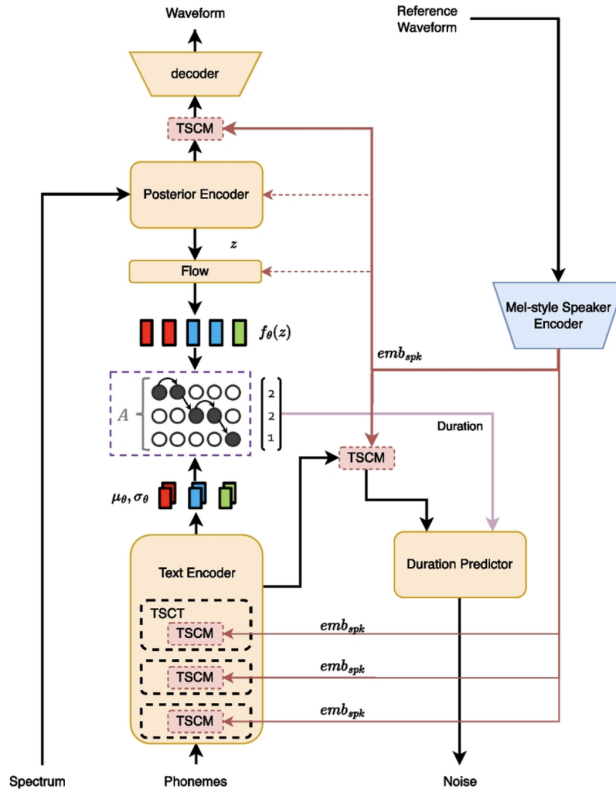


Figure 2.4: VITS Architecture

2.8.2 Training Objective

VITS is trained end-to-end using a combination of variational lower bound loss, adversarial training loss from a multi-period discriminator, and feature matching loss. This joint training allows the model to simultaneously optimize acoustic quality and waveform naturalness.

2.8.3 Nepali-Specific Configuration for VITS

- VITS was pre-trained on the LJSpeech dataset [6] to initialize strong acoustic priors before fine-tuning on the Nepali dataset, leveraging transfer learning to compensate for limited Nepali training data.
- The input phoneme set was adapted to include Nepali phonemes, with the model’s phoneme embedding layer extended accordingly.
- Fine-tuning hyperparameters including learning rate scheduling and batch size — were adjusted to prevent catastrophic forgetting of the pre-trained acoustic knowledge while adapting to Nepali speech characteristics.
- Multiple fine-tuning configurations were tested to identify the optimal balance between LJSpeech pre-training and Nepali adaptation.

2.9 SpeechT5

SpeechT5 [4] is a unified sequence-to-sequence framework that extends the T5 [22] text-to-text paradigm to speech and language tasks. Pre-trained on large quantities of both labelled and unlabelled speech and text data, SpeechT5 supports multiple speech processing tasks including TTS, ASR, and voice conversion within a single shared encoder-decoder architecture.

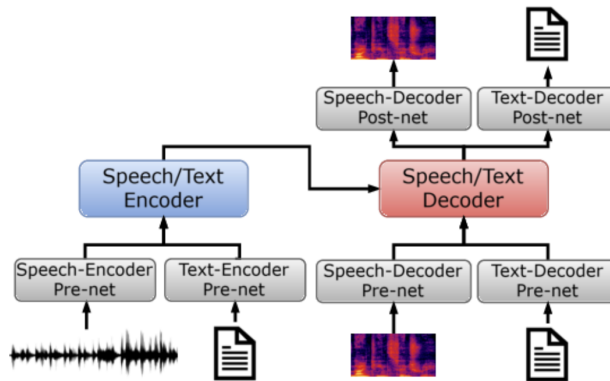


Figure 2.5: SpeechT5 Architecture

2.9.1 Architecture

- **Shared Encoder-Decoder:** A transformer-based encoder-decoder backbone is shared across all tasks. For TTS, the encoder processes text input and the decoder generates speech output, conditioned on a speaker embedding.
- **Pre-Net and Post-Net:** Task-specific pre-processing and post-processing network modules are attached to the shared backbone for each modality. For TTS, a pre-net converts text tokens into embeddings, and a post-net refines the predicted mel-spectrograms before vocoding.
- **Speaker Embedding:** A speaker representation vector is injected into the decoder to condition the output on a specific speaker identity, enabling speaker-adaptive synthesis.
- **Cross-Modal Pre-Training:** SpeechT5 is pre-trained jointly on speech and text corpora, allowing the model to leverage linguistic knowledge from text to improve speech synthesis quality — particularly beneficial for low-resource languages where labelled speech data is limited.

2.9.2 Nepali-Specific Configuration for SpeechT5

- SpeechT5 was initialized from Microsoft’s publicly available pre-trained checkpoint, pre-trained on English speech and text data, before fine-tuning on the Nepali dataset.
- A Nepali speaker embedding was extracted from the training dataset and used to condition the decoder throughout fine-tuning.
- The tokenizer vocabulary was extended to include Nepali Devanagari characters, ensuring full coverage of the script without reliance on transliteration.

- Fine-tuning was conducted with careful learning rate scheduling to preserve the cross-modal linguistic knowledge acquired during pre-training while adapting the model to Nepali phonetic and prosodic characteristics.

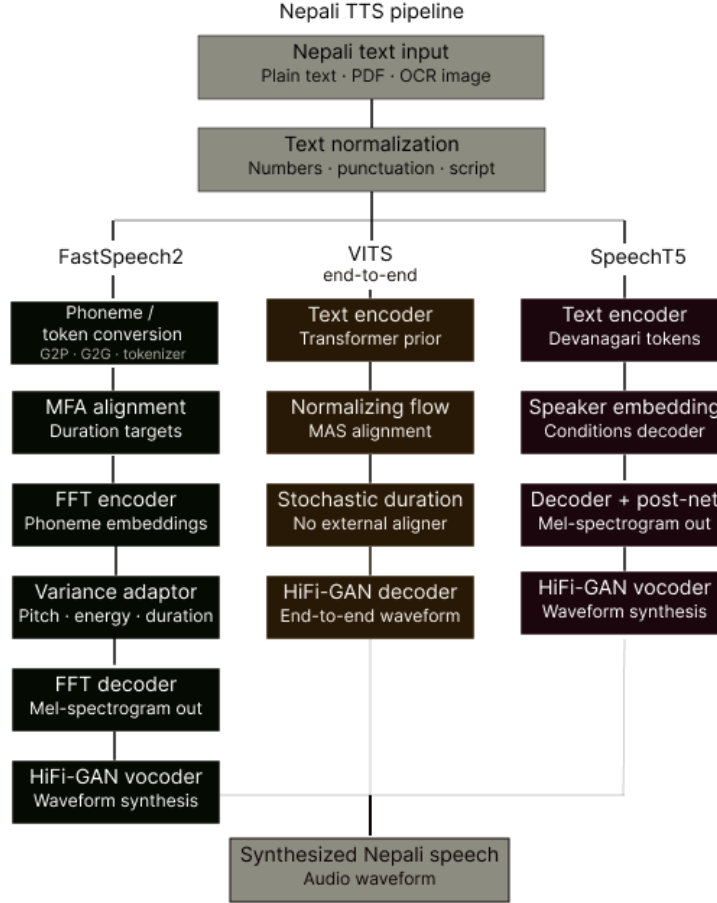


Figure 2.6: TTS system pipeline comparison across FastSpeech2, VITS, and SpeechT5

2.10 Vocoder: Mel-Spectrogram to Waveform Synthesis

The vocoder module converts mel-spectrograms into audible speech waveforms. For FastSpeech2 and SpeechT5, which produce explicit mel-spectrograms as output, a separate neural vocoder is required to complete the synthesis pipeline. VITS, by contrast, incorporates waveform generation directly within its decoder and does not require an external vocoder.

The evolution of vocoder technologies reflects the broader progress in TTS systems:

- **Traditional approaches:** Methods such as Griffin-Lim relied on iterative phase reconstruction from magnitude spectrograms but produced low naturalness output.
- **Neural vocoders:** Revolutionized waveform generation by directly learning the mapping from spectrograms to waveforms through data-driven training.

2.10.1 HiFi-GAN

HiFi-GAN [23] is a GAN-based neural vocoder optimized for high-fidelity, real-time waveform synthesis. It employs a generator paired with multi-period and multi-scale discriminators, enabling the model to capture both fine-grained local acoustic patterns and broader global signal structure simultaneously. HiFi-GAN has become the standard vocoder choice in modern TTS pipelines due to its excellent perceptual quality and fast inference speed, and was used as the vocoder for both FastSpeech2 and SpeechT5 in this project.

Table 2.1: Summary of Neural Vocoders

Vocoder	Type	Inference	Quality
WaveNet	Autoregressive	Slow	Very High
WaveGlow	Flow-based	Fast (Parallel)	High
HiFi-GAN	GAN-based	Real-time	Excellent

Recent studies demonstrate that combining FastSpeech2 with HiFi-GAN achieves competitive performance in terms of both Mean Opinion Score (MOS) and inference latency, particularly in low-resource linguistic environments [12].

2.11 Evaluation Metrics for the TTS System

Evaluating a Text-to-Speech (TTS) system requires both objective and subjective metrics to determine how natural, intelligible, and accurate the synthesized speech is compared to real human speech. For the Nepali TTS system, the following metrics are commonly used to assess performance:

- **Mel Cepstral Distortion (MCD):** Mel Cepstral Distortion (MCD) is an objective measure used to quantify the difference between the spectral features of synthesized and natural speech. It evaluates how closely the generated speech waveform’s spectral envelope matches that of the reference (ground truth) signal. MCD is calculated based on the Mel-Frequency Cepstral Coefficients (MFCCs), which represent the short-term power spectrum of sound. A lower MCD value indicates that the synthesized speech is more similar to the natural reference, implying better spectral quality. In Nepali TTS evaluation, MCD helps assess how accurately the model captures the tonal and vowel-rich characteristics of the Nepali language.

A lower F0-RMSE value indicates that the model more accurately captures the pitch pattern and rhythm of the original speaker. This is especially important for Nepali TTS, as proper pitch contouring affects the perceived naturalness and emotional tone of synthesized Nepali speech.

- **Mean Opinion Score (MOS):** The Mean Opinion Score (MOS) is a subjective metric obtained by collecting human listeners’ opinions on the quality of synthesized speech. Participants rate samples on a 5-point scale (typically from 1 – “Bad” to 5 – “Excellent”) based on naturalness, clarity, and similarity to human voice. MOS provides insights into how real users perceive the TTS system’s performance, which objective metrics alone cannot capture. For Nepali speech synthesis, MOS evaluation helps determine whether the voice sounds natural, pleasant, and contextually appropriate to native Nepali listeners.

Table 2.2: Other Evaluation Metrics for Nepali TTS System

Metric	Type	Measures	Interpretation
MCD	Objective	Spectral similarity	Lower = closer to natural speech
F0-RMSE	Objective	Pitch contour accuracy	Lower = better pitch reproduction
WER / CER	Objective	Intelligibility	Lower = higher clarity & recognition
MOS	Subjective	Naturalness, clarity, similarity	Higher = more natural & pleasant

2.12 Nepali Language and Devanagari Script

Nepali is an Indo-Aryan language spoken by approximately 17 million people as a native language, primarily in Nepal, with additional speakers in India, Bhutan, and Myanmar [24]. It is the official language of Nepal and serves as the lingua franca across the country’s diverse linguistic communities. Nepali is written in the Devanagari script, a left-to-right abugida in which consonants carry an inherent vowel sound that is modified or suppressed through the use of diacritical marks. Understanding the structure of Devanagari and the phonological rules of Nepali is essential for the design of any robust text processing pipeline for TTS.

2.12.1 Structure of the Devanagari Script

Devanagari is an *abugida*, also called an alphasyllabary, meaning that each base character represents a consonant with an inherent vowel, and additional diacritical marks are used to modify or cancel that vowel. The script is used to write several South Asian languages including Hindi, Sanskrit, Marathi, and Nepali, with minor variations in character inventory across languages.

The Devanagari character set as used in Nepali consists of the following categories:

- **Swar (Vowels):** Independent vowel characters used at the beginning of a syllable or word. Nepali uses the following independent vowels:

अ	आ	इ	ई	उ	ऊ	ए	ऐ	ओ	औ	अं
a	aa	i	ii	u	uu	e	ai	o	au	am

- **Vyanjan (Consonants):** The consonant inventory of Nepali Devanagari includes 36 base consonants, organized traditionally by their place and manner of articulation:

क	ख	ग	घ	ङ
च	छ	ज	झ	ञ
ट	ठ	ड	ढ	ण
त	थ	द	ध	न
प	फ	ब	भ	म
य	र	ल	व	—
श	ष	स	ह	—

- **Matra (Vowel Diacritics):** When a vowel follows a consonant, it is written not as an independent character but as a diacritical mark attached to the consonant, called a matra. Each independent vowel (except *a*) has a corresponding matra form:

Vowel	Matra	Example
आ (aa)	ा	का (kaa)
इ (i)	ि	कि (ki)
ई (ii)	ी	की (kii)
उ (u)	ु	कु (ku)
ऊ (uu)	ू	कू (kuu)
ए (e)	े	के (ke)
ऐ (ai)	ै	कै (kai)
ओ (o)	ो	को (ko)
औ (au)	ौ	कौ (kau)

- **Anusvara (अं):** A nasal diacritic written as a dot above the character, representing nasalization of the preceding vowel. For example, संस्कार (samskaar). In Nepali TTS preprocessing, the anusvara must be resolved to the appropriate nasal consonant depending on the following consonant's place of articulation.
- **Visarga (अः):** A diacritic written as two dots after a vowel (◌ः), representing an aspiration or breathy voice release. It is relatively rare in colloquial Nepali but appears in Sanskrit-origin words.
- **Chandrabindu (अँ):** A diacritic representing pure nasalization of the vowel, distinct from the anusvara. For example, आँखा (aankhaa, meaning eye).
- **Halanta (Virama):** The halanta, written as ् below a consonant, explicitly suppresses the inherent vowel of that consonant. It is one of the most important characters in Devanagari processing, discussed in detail in Section 2.12.3.
- **Nukta (◌◌):** A dot placed below a consonant to represent sounds borrowed from Persian and Arabic, such as क़, ख़, ग़. These characters are rare in standard Nepali but may appear in loanwords.

2.12.2 Unicode Encoding of Devanagari

In digital systems, Devanagari text is encoded using the Unicode standard. The Devanagari Unicode block spans the range U+0900 to U+097F. Key code points relevant to Nepali TTS processing are listed in Table 2.3.

Table 2.3: Key Unicode Code Points for Nepali Devanagari

Character	Unicode	Description
अ	U+0905	Independent vowel A
क	U+0915	Consonant KA
्	U+094D	Halanta (Virama)
ा	U+093E	Matra AA
ि	U+093F	Matra I
ं	U+0902	Anusvara
ः	U+0903	Visarga
ँ	U+0901	Chandrabindu
◌◌	U+093C	Nukta

A critical aspect of digital Devanagari processing is that a single *visually rendered* syllable may be composed of multiple Unicode code points. For example, the syllable कि is stored as two code points: U+0915 (क) followed by U+093F (ि). This means that naive character-level string operations in Python or any programming language can produce incorrect results when applied to Devanagari text, and dedicated Unicode-aware libraries must be used.

2.12.3 Conjunct Consonants and the Halanta

When two or more consonants appear in sequence without an intervening vowel, they form a *conjunct consonant* (yuktakshara). In Devanagari, conjunct consonants are formed by joining the base consonants using the halanta (्, U+094D), which suppresses the inherent vowel of the preceding consonant.

For example:

- क + ् + त = क्त (kta), as in शक्ति (shakti)
- प + ् + र = प्र (pra), as in प्रकाश (prakaash)
- स + ् + क = स्क (ska), as in स्कूल (skuul)

In rendering engines, the halanta typically triggers a ligature or half-form of the preceding consonant rather than being visually displayed as the ् character itself. However, in the underlying Unicode text stream, the halanta code point is always present between the two consonant code points. For TTS preprocessing, this is significant because the phoneme sequence must correctly reflect the absence of a vowel between the two consonants.

When a halanta appears at the end of a word with no following consonant, it indicates a consonant-final syllable, meaning the inherent vowel of the final consonant is explicitly suppressed. This is important for correct pronunciation at word boundaries.

2.12.4 The Schwa and Schwa Deletion in Nepali

The most phonologically significant challenge in Devanagari-based TTS is the treatment of the *schwa*, the inherent vowel /ə/ (or /a/ in Nepali) that is implicitly present after every consonant that is not followed by a matra or halanta. While the script does not write this vowel explicitly, it is nonetheless pronounced in most positions. However, in certain environments, the schwa is deleted in natural spoken Nepali, a phenomenon known as *schwa deletion* or *schwa syncope*.

Schwa deletion is one of the most important rules to implement correctly in any Nepali or Hindi TTS system because an incorrect schwa leads to perceptibly unnatural pronunciation. Consider the word कमल (ka-mal, meaning lotus). Naively applying the inherent vowel to every consonant would yield the pronunciation *ka-ma-la*, but the correct Nepali pronunciation is *ka-mal*, where the final schwa is deleted.

Rules for Schwa Deletion in Nepali

The following rules govern schwa deletion in Nepali, adapted from established phonological analyses of Hindi-Nepali Devanagari [25, 26]:

1. **Word-final schwa deletion:** The inherent vowel of the final consonant in a word is almost always deleted in standard Nepali pronunciation.

राम → /raam/ (not /raama/)

काम → /kaam/ (not /kaama/)

घर → /ghar/ (not /ghara/)

2. **Medial schwa deletion before a single consonant followed by a vowel:** When a consonant in the middle of a word is followed by another consonant that itself is followed by an explicit vowel, the schwa after the first consonant is deleted.

समझ → /samajh/ (not /sa-ma-jha/)

3. **Schwa retention before conjunct consonants:** When a consonant precedes a conjunct (two consonants joined by halanta), the schwa before the conjunct is typically retained to maintain syllable structure.

विकल्प → /vikalpa/ (schwa retained before क्ल)

संस्कृत → /sanskrit/ (complex conjunct cluster)

4. **Schwa retention with anusvara:** When a consonant is followed by an anusvara (◌ं), the schwa preceding the anusvara is retained.

संसार → /samsaar/ (schwa retained after स)

5. **Schwa deletion before word-medial halanta:** A schwa is deleted when the following consonant carries a halanta, as the halanta already suppresses the following inherent vowel.

सत्य → /satya/ (not /sa-ta-ya/)

2.12.5 Text Normalization for Nepali

Before a Nepali text string can be passed to the G2P module, it must be normalized to resolve all non-standard textual elements into their speakable forms. The following normalization steps are applied in the preprocessing pipeline of this project:

1. **Numeral conversion:** Devanagari uses its own numeral system (०, १, २, ३, ४, ५, ६, ७, ८, ९) alongside Arabic numerals (0--9). Both forms must be detected and converted to their spoken Nepali equivalents. For example, २०८२ is normalized to दुई हजार बयासी.
2. **Punctuation handling:** Punctuation marks such as the Devanagari danda (।, U+0964) and double danda (॥, U+0965), which serve as sentence terminators, are mapped to prosodic pause markers. Standard punctuation such as commas and full stops are similarly mapped.
3. **Abbreviation expansion:** Common Nepali abbreviations such as डा. (Doctor) and श्री (Shri) are expanded to their full spoken forms.
4. **Whitespace and encoding normalization:** Zero-width non-joiner (ZWNJ, U+200C) and zero-width joiner (ZWJ, U+200D) characters, which are sometimes inserted in Devanagari text to control ligature formation, are stripped or handled appropriately. Unicode normalization form NFC is applied to ensure consistent composition of characters and diacritics.
5. **Mixed script handling:** Nepali digital text often contains English words or numerals embedded within Devanagari text. Such tokens are either transliterated into Devanagari pronunciation or handled by a separate English G2P module.

2.12.6 Tokenization and Syllabification

Following normalization, the text is tokenized into a sequence of syllables or phoneme clusters. Nepali syllabification follows the Devanagari structure where a syllable is defined as a consonant cluster (with any conjunct consonants resolved via halanta) followed by a vowel (either the inherent schwa or an explicit matra). The syllabified output, after schwa deletion is applied, forms the input to the G2P conversion module, which maps each syllable to its corresponding phoneme sequence for the acoustic model.

2.12.7 Transliteration

For Nepali TTS systems such as SpeechT5 and other neural speech synthesis architectures, transliteration is commonly used as an intermediate representation to convert Nepali Devanagari script into a standardized Romanized phonetic form. This process enables easier integration with speech processing tools, pronunciation lexicons, alignment frameworks, and phoneme-based modeling pipelines that are often designed primarily for Latin-script inputs.

In our work with SpeechT5, transliteration was used to convert Nepali Devanagari text into a consistent Romanized representation that preserves important phonetic characteristics of the language, including aspirated and unaspirated consonants, vowel distinctions, nasal sounds, and special modifiers.

Nepali	Transliteration	Nepali	Transliteration	Nepali	Transliteration	Nepali	Transliteration
आइ	aaj	औ	Sow	क	kax	व	wax
ाइ	Saaj	ेइ	Sej	ख	khax	श	sax
ओइ	oj	औ	aw	ग	gax	ष	sax
ोइ	Soj	ी	Saw	ज	nax	स	sax
ओई	oj	आ	aa	ट	ttax	ह	hax
ोई	Soj	ा	Saa	ठ	tthax	ं	m
एई	ej	इ	i	ड	ddax	ँ	n
ेई	Sej	ि	Si	ढ	ddhax	ः	
उइ	uj	ई	i	ण	nnax	ऴ	\$
ुइ	Suj	ी	Si	त	tax	इउ	iw
उई	uj	उ	u	थ	thax	िउ	Siw
ुई	Suj	ू	Su	द	dax	इक	iw
आउ	aaaw	ऊ	u	ध	dhax	िक	Siw
ाउ	Saaw	ू	Su	न	nax	आई	aaj
आऊ	aaaw	ए	e	प	pax	ाई	Saaj
ाऊ	Saaw	े	Se	फ	phax	एइ	ej
एउ	ew	ओ	o	ब	bax	घ	ghax
ेउ	Sew	ी	So	भ	bhax	ङ	Ngax
एऊ	ew	ऋ	ri	म	max	च	Chax
ेऊ	Sew	ृ	Sri	य	yax	छ	chhax
ओउ	ow	ऐ	axj	र	rax	ज	Jhax
ोउ	Sow	ै	Saxj	ल	lax	झ	jhhax
ओऊ	ow	अ	ax				

Figure 2.7: Nepali transliteration example used in the preprocessing pipeline

Since many existing TTS preprocessing pipelines and alignment tools have limited native support for Nepali, the transliteration layer provides a standardized and machine-readable representation that improves compatibility across multiple models, datasets, and experimental setups. It also enabled consistent phonetic processing during training and inference for Nepali speech synthesis.

3. Methodology

3.1 Feasibility Study

The feasibility study was conducted to assess the technical, operational, and economic viability of developing an end-to-end Nepali TTS system.

- **Technical Feasibility:**

The proposed system requires three technically distinct capabilities to operate as an integrated pipeline: extraction of Nepali text from physical or digital documents, conversion of that text into natural-sounding speech, and sequential delivery of audio output to the end user. Each component was assessed independently for technical viability prior to system development.

Text Extraction: Optical character recognition of Devanagari script presents well-documented challenges, including complex ligature structures, conjunct consonants (*halanta* forms), and high inter-glyph similarity. Tesseract OCR, actively maintained by Google, provides a dedicated trained model for Devanagari script that has demonstrated reliable character-level accuracy on clean printed Nepali text. Its permissive open-source license, support for custom model training, and active community maintenance establish it as a technically adequate and low-risk choice for the document extraction component, provided that input document quality is reasonably controlled. Recognition of degraded prints or handwritten Nepali text is acknowledged as a known boundary of the tool’s reliable capability.

Speech Synthesis: Nepali is a low-resource language with a limited volume of publicly annotated speech data, which makes training a very high-quality, natural sounding TTS model entirely from scratch technically infeasible within the scope of this project. The viability of the synthesis component therefore depends on the availability of pre-trained speech models that can be effectively adapted to Nepali through fine-tuning on a smaller target-language corpus. Among non-autoregressive architectures, FastSpeech 2 is a strong candidate due to its parallel inference, stable training dynamics, and expressive prosody control through explicit duration, pitch, and energy predictors. VITS, an end-to-end flow-based model, offers the additional advantage of eliminating a separate vocoder by generating waveforms directly, though it generally demands larger training sets to achieve stable convergence. For autoregressive approaches, SpeechT5, a transformer encoder-decoder model pre-trained on large-scale English speech corpora presents a particularly viable path as its pre-trained acoustic representations provide a strong general-purpose foundation that requires only fine-tuning on Nepali data rather than training from scratch, substantially reducing the target-language data requirement. A Devanagari-to-Roman transliteration layer can bridge the script mismatch between Nepali input and the model’s existing tokenizer, making end-to-end synthesis achievable without retraining the tokenizer. HiFi-GAN, a neural vocoder with a well-established track record of high-fidelity mel-spectrogram-to-waveform conversion, is technically suitable to serve as the final synthesis stage across these architectures.

A comparative evaluation of these architectures is planned to identify the configuration that yields the best synthesis quality and training stability on the available Nepali corpora.

Data Adequacy: The availability of labelled Nepali speech data is the primary technical risk of

this project. The OpenSLR Nepali corpus and the IndicTTS Nepali dataset are the most substantial publicly available resources, each providing transcribed read speech with coverage across a range of phonemes and prosodic patterns. While neither corpus is large by contemporary TTS standards, the transfer learning strategy proposed above significantly reduces the volume of target-language data needed to achieve acceptable synthesis quality, making the project technically feasible within these data constraints.

Compute and System Integration: Fine-tuning transformer-based TTS models requires GPU acceleration. Free-tier cloud environments, specifically Kaggle Notebooks and Google Colaboratory, provide access to NVIDIA GPU hardware at no cost, establishing that the training phase is executable without dedicated infrastructure. At inference time, the pipeline comprising OCR, transliteration, TTS inference, and vocoding , operates sequentially and imposes no hard real-time constraints that would require hardware beyond a standard GPU-equipped machine. This confirms that the integrated system is deployable within accessible compute environments.

Taken together, the availability of suitable open-source tools, publicly accessible Nepali speech corpora, and accessible cloud compute resources establishes that the proposed system is technically feasible. The principal technical risk- synthesis quality under low-resource data conditions , is structurally mitigated by the adoption of a transfer learning approach.

- **Operational Feasibility:**

The system is designed to accept standard printed Nepali text and produce intelligible synthesised speech, requiring no specialised hardware or technical literacy from the end user beyond a standard computing device. Its primary beneficiaries (visually impaired individuals, students, and general readers) represent a clearly underserved population for whom no accessible Nepali TTS alternative currently exists. The pipeline is deliberately scoped to exclude mixed-language input, handwritten text, and real-time translation, keeping operational complexity manageable and failure modes limited. Being composed entirely of modular, open-source components, the system can be maintained and updated incrementally without requiring dedicated institutional infrastructure.

- **Economic Feasibility:**

All core system components including Tesseract OCR, pre-trained TTS model weights, the HiFi-GAN vocoder, and associated training frameworks are available under open-source licences, eliminating software costs entirely. Model training and experimentation are carried out on free-tier cloud platforms, specifically Kaggle Notebooks and Google Colaboratory, which provide GPU acceleration at no cost, removing the need for dedicated hardware investment. The only anticipated variable cost is the potential procurement of additional curated Nepali speech data should the available public corpora prove insufficient, and hosting costs if the system is eventually deployed as a public service. Against these minimal expenditures, the system delivers accessibility value to a population currently underserved by existing TTS solutions, making the project economically viable with a strongly favourable cost-to-impact ratio.

3.2 Requirement Analysis

1. Functional Requirements

- Accept Nepali digitized text as input
- Extract, normalize and preprocess the input text from image
- Allow multi model Text-to-Speech synthesis(FastSoeech2, VITS, SpeechT5)
- Provide audio playback and download

2. Non-Functional Requirements:

- Naturalness: Maintain human-like prosody and tone.
- Efficiency: Achieve low-latency synthesis for real-time applications with acceptable system responsiveness.
- Scalability: Enable future expansion to multi-speaker or multilingual setups.
- Portability (runs on CPU as fallback)

3. Hardware Requirements:

- Training: Kaggle/Colab free tier GPUs (T4, P100)
- Inference: minimum CPU/RAM specs for deployment

4. Software Requirements

- Python, PyTorch, NumPy, librosa
- Pretrained model dependencies

5. Dataset Requirements

- OpenSLR SLR43
- IndicTTS Nepali dataset
- LJSpeech (for fine-tuning)

3.3 Methodology Pipeline

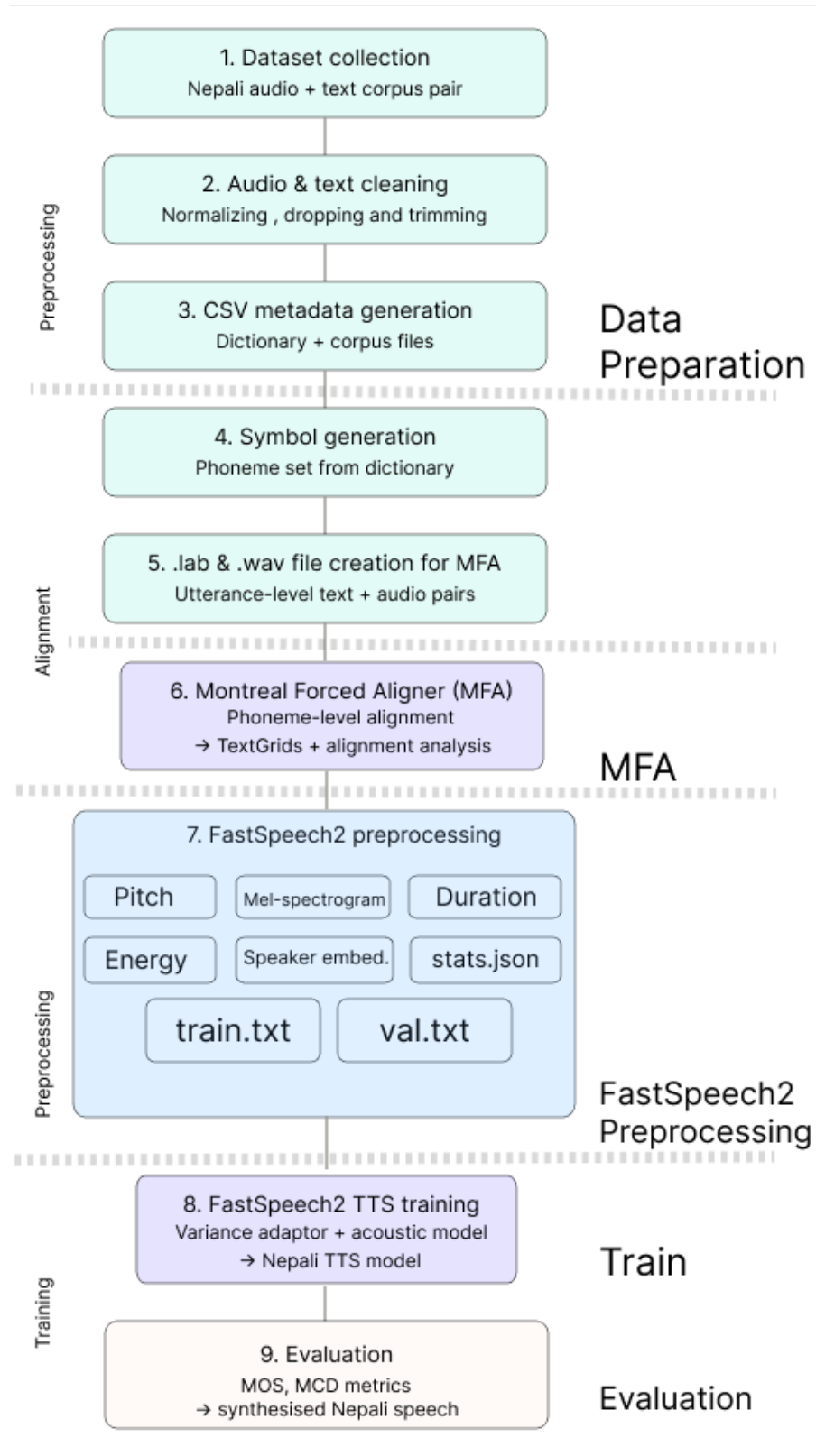


Figure 3.1: Methodology of the FastSpeech2 - Nepali TTS pipeline from scratch.

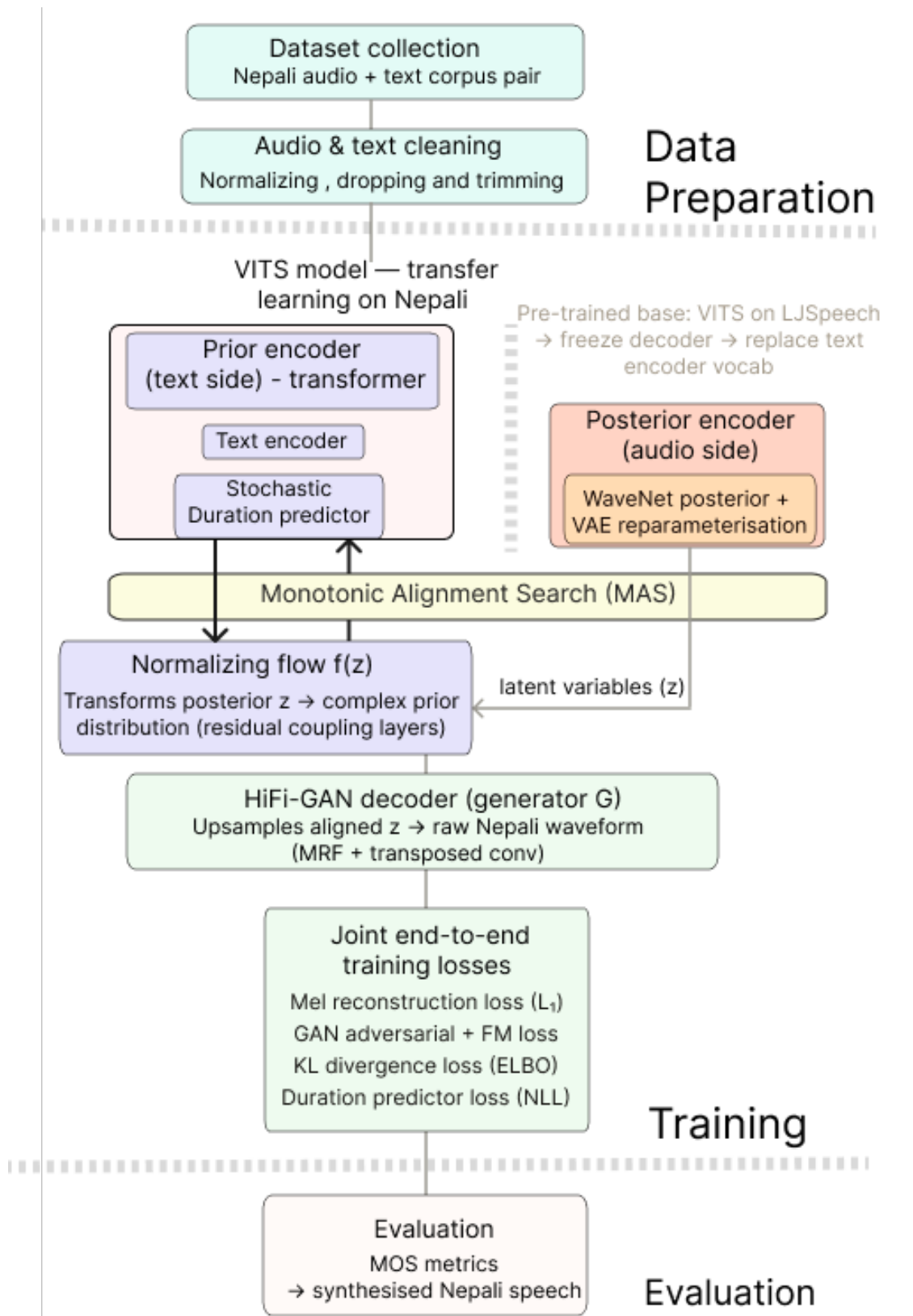


Figure 3.2: Methodology of the VITS - Nepali TTS pipeline using Transfer Learning.

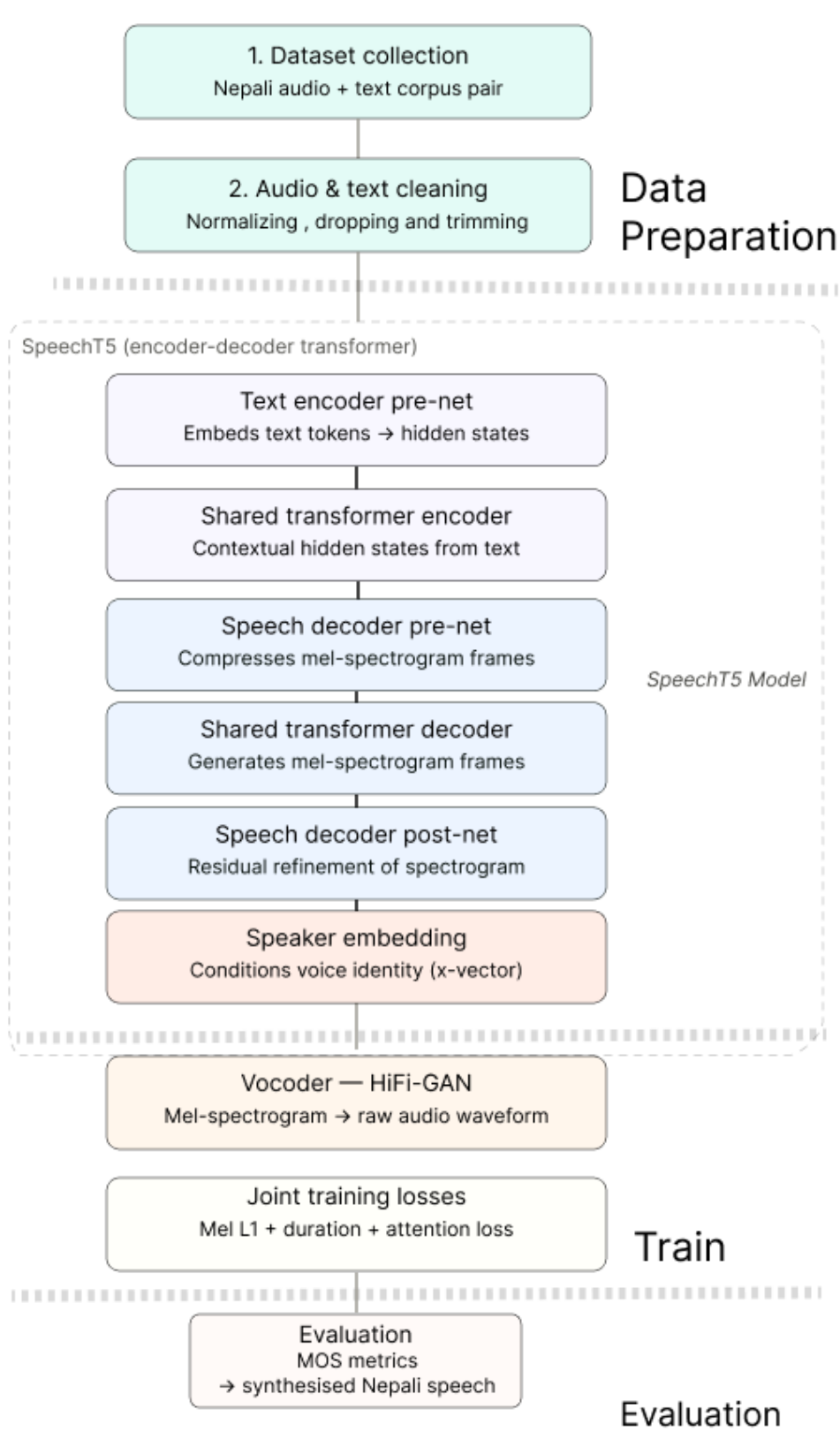


Figure 3.3: Methodology of the SpeechT5 - Nepali TTS pipeline using Transfer Learning.

3.3.1 Data Collection

Publicly available Nepali speech datasets were utilized in this research to ensure reproducibility and reliable speech quality. Two primary datasets from the OpenSLR repository and IndicTTS resources were used: SLR43 and IndicTTS. And a small dataset curated by a senior was borrowed for testing as well.

- **OpenSLR SLR43:**

The “High-Quality Nepali TTS Dataset” (SLR43) consists of recordings from 19 native female speakers, containing a total of 2,064 utterances (approximately 2.8 hours of audio). Developed by Google in Nepal, the dataset provides high-fidelity 48 kHz, 16-bit mono recordings aligned with accurately transcribed Nepali text. The speech material includes literary readings, news articles, and conversational phrases, offering phonetic and prosodic diversity.

For this project, the dataset was preprocessed to standardize the training pipeline. Audio files were downsampled to 22.5kHz and a consistent format, transcripts were normalized, and metadata files were generated to align speech with textual input. Due to its clean recordings and high-quality annotations, SLR43 served as the initial dataset for baseline experiments.

- **IndicTTS Nepali Dataset:**

In addition to SLR43, a Nepali subset from the IndicTTS corpus was utilized to provide a larger dataset for extended experimentation. After preprocessing, the dataset contained 4,947 utterances with variable audio lengths, totaling roughly 3 GB of data and around 10.32 hours of speech. The dataset primarily consisted of mono recordings from a female speaker. Similar preprocessing steps were applied, including transcript normalization, segmentation verification, and conversion of audio files into a consistent format suitable for model training.

Compared to SLR43, the IndicTTS dataset provided a significantly larger volume of speech data, making it suitable for experiments requiring longer training durations and improved acoustic coverage. However, the dataset contained substantial inconsistencies in word spellings, along with heavy contamination from Hindi and Sanskrit orthographic conventions, including the frequent use of Nukta-based characters and non-standard symbols not typically used in Nepali writing.

- **Self-Curated Studio Recording Dataset:**

To supplement the publicly available datasets, a small self-curated Nepali speech dataset was also utilized. The dataset consisted of approximately 1 hour of studio-quality recordings collected from a single native male speaker. The recordings were professionally captured and shared by a senior collaborator, providing clean audio with minimal background noise and consistent speaking conditions.

The dataset primarily served as an experimental resource for evaluating speaker-specific synthesis quality and observing model behavior on high-fidelity single-speaker recordings. Similar preprocessing procedures were applied, including transcript normalization, silence trimming, sampling rate standardization, and metadata generation for compatibility with the training pipeline.

The initial stage of the project focused on evaluating how a Nepali Text-to-Speech (TTS) system behaves when trained with limited language resources. To investigate this, both SLR43 and IndicTTS datasets were first used to train models from scratch without relying on pretrained multilingual models. This allowed observation of model convergence, pronunciation accuracy, and mel-spectrogram alignment under low-resource conditions.

This two-stage methodology : training from scratch followed by fine-tuning with a pretrained model enabled a comparative analysis of:

- Training Nepali TTS systems with limited datasets.
- The benefits of larger speech corpora for model convergence.
- The effectiveness of transfer learning for low-resource languages.

Table 3.1: Summary of Nepali Speech Datasets Used

Dataset	Speakers	Duration (hrs)	Utterances	Gender
SLR43	19	2.8	2,064	Female only
IndicTTS (Nepali)	Single	10.32	4,950	Female
Self-Curated Studio Dataset	Single	1.0	300	Male

3.3.2 Data Preprocessing

With FastSpeech2, data cleaning and preprocessing pipeline prepares textual and audio data for Montreal Forced Aligner. The goal is to ensure data uniformity, linguistic consistency, and high-quality audio-text alignment, which is critical for the TTS performance.

In the case of VITS, preprocessing similarly focuses on transcript normalization, phoneme consistency, and audio standardization; however, the model does not explicitly rely on externally generated alignments from MFA. Instead, VITS learns latent alignments internally through its monotonic alignment mechanism during end-to-end training, reducing dependence on manually aligned duration information.

For SpeechT5, preprocessing emphasizes text normalization, tokenizer compatibility, and consistent audio feature extraction suitable for pretrained transformer-based architectures. Since SpeechT5 leverages transfer learning from large-scale pretrained representations, maintaining consistent input formatting and high-quality paired speech-text data becomes essential for effective fine-tuning and adaptation to Nepali speech synthesis.

Common Text Normalization and Preprocessing

Text normalization and preprocessing were carried out to prepare consistent and model-friendly transcripts from the metadata of text–audio pairings. These steps ensured that the textual input aligned properly with the acoustic representations used during model training. The preprocessing pipeline consisted of the following stages:

1. **Unicode Normalization:** All characters were converted to Unicode NFC form to maintain consistent encoding across the dataset. This step was essential for debugging, reproducibility, and ensuring that visually identical characters shared the same underlying representation.
2. **Number, Date, and Currency Expansion:** Numerals, years, and date formats appearing in transcripts were expanded into their full Nepali word forms using a custom Python script. This ensured that the model learned spoken forms rather than numerical representations.

3. **Removal of Unwanted Characters:** Non-standard symbols, foreign characters, and unseen Unicode symbols were removed from the transcripts to prevent unexpected pronunciation errors during synthesis.
4. **Abbreviation Expansion:** Common abbreviations were expanded into their complete Nepali equivalents to improve clarity and pronunciation consistency during training.
5. **Grapheme Decomposition Experiments:** Several decomposition approaches were explored for FastSpeech2 model for representing Nepali graphemes during preprocessing. Initially, a fully decomposed representation was tested where matras (vowel signs), halants (virama), and base consonants were separated into individual symbols. This approach ensured that every character component was represented explicitly and consistently in the training data.

In later experiments, a cluster-based representation was tested. Instead of fully decomposing graphemes, conjunct consonants were handled as clusters within a pronunciation dictionary. Complex conjunct words were broken down into phonetic or grapheme clusters with the expectation that the model would better learn pronunciation patterns for frequently occurring conjunct structures in Nepali.

Table 3.2: Comparison of Grapheme-Based and Cluster-Based Representation in Nepali

Word	Grapheme-Based Separation	Cluster-Based Separation
कृपया	कृ प य ा	कृ प या
स्पष्टीकरण	स् प ष ट् टी क र ण	स् प ष्ठी क र ण
छोरलाई	छो र ा ल ा ई	छो रा लाई
नेपाल	ने प ा ल	ने पा ल

These experiments were conducted to evaluate how different levels of grapheme decomposition affected pronunciation accuracy, handling of conjunct characters, and overall stability of the text-to-speech model.

Common Audio Preprocessing

Audio processing was conducted to ensure uniformity and reduce artifacts that can degrade model performance. Steps included:

- Resampling all audio to **22.05 kHz, 16-bit, mono** format.
- Trimming silence at the beginning and end of recordings.
- Segmenting long recordings into 2–12 second clips for optimal model training.
- Filtering out noisy or low-quality recordings based on energy and signal-to-noise ratio.

3.3.3 FastSpeech2 : Forced Alignment and Feature Preparation with Preprocessing

A critical stage in all the Nepali TTS pipelines involved aligning the cleaned audio recordings with their corresponding textual transcriptions. While the VITS and SpeechT5 had inbuilt encoders/aligners, the alignment for FastSpeech2 pipeline required a lot of manual testing and training of the MFA. After completing the audio and text cleaning stages, the dataset was prepared for grapheme-level alignment. This alignment was performed using the Montreal Forced Aligner (MFA), a widely used toolkit for phoneme-level speech alignment. Forced alignment is essential in neural TTS systems because it determines the temporal relationship between linguistic units (phonemes or graphemes) and their acoustic realizations. Accurate alignment enables the model to learn phoneme durations, prosodic timing, and speech rhythm, which are necessary for producing natural-sounding synthesized speech.

Preparation of Alignment Files

After text normalization and audio cleaning, each utterance was paired with its transcription by generating individual .lab files. These files contained the normalized text corresponding to each .wav file and shared the same filename identifier. This structure allowed the aligner to correctly associate speech signals with their textual representations. The dataset directory therefore contained:

Table 3.3: Input File Structure for Forced Alignment

File Type	Description
.wav	Raw audio recording of the utterance
.lab	Normalized text transcription of the audio
dictionary.txt	Grapheme-to-phoneme pronunciation dictionary

Montreal Forced Aligner (MFA)

The prepared dataset was then processed using the Montreal Forced Aligner (MFA).

During alignment, MFA used the provided pronunciation dictionary and acoustic modeling to estimate the temporal boundaries of phonetic units within each audio file. The alignment process produced the following outputs:

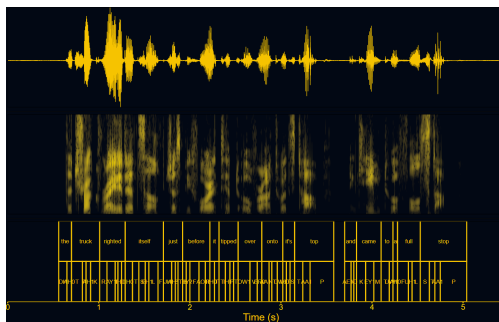


Figure 3.4: Montreal Forced Aligner

- **TextGrid files:** These files contained time-aligned annotations of phonemes or graphemes for each utterance, specifying the start and end time of each unit within the waveform. Each TextGrid file contains hierarchical tiers consisting of intervals that represent phonemes, words, or silence segments. Each interval specifies the start time, end time, and corresponding textual unit. These alignments play several important roles in the TTS training pipeline:
 - Extraction of phoneme-level duration targets for the FastSpeech2 duration predictor.
 - Accurate extraction of pitch and energy values at phoneme boundaries.
 - Ensuring temporal consistency between linguistic input and acoustic features.
- **Alignment analysis:** MFA also generated diagnostic information that helped identify problematic alignments, missing pronunciations in the dictionary, and mismatches between text and speech.

The generated TextGrid alignments served as the foundation for extracting duration information required by the TTS model. Without reliable alignment, the model would be unable to learn proper timing relationships between text and speech, resulting in unnatural prosody and incorrect speech rhythm. Additional preprocessing operations included:

- Removal of corrupted or incomplete TextGrid files
- Realigning the MFA Training
- Normalization of pitch and energy ranges
- Validation of consistent feature dimensions
- Logging of processed and skipped samples

Following successful alignment, the preprocessing pipeline of the FastSpeech2 TTS model was executed. This stage utilized the aligned transcripts, pronunciation dictionary, and corresponding audio files to extract acoustic features necessary for model training. The preprocessing stage generated several key features from each text-audio pair:

Table 3.4: Acoustic Features Extracted for FastSpeech2 Training

Feature	Description
Mel-Spectrogram	Time-frequency representation of the speech signal
Pitch (F0)	Fundamental frequency capturing tonal variations
Energy	Frame-level amplitude representing speech loudness
Phoneme Duration	Time duration of each phoneme from MFA alignment

These extracted features were stored as intermediate training files and served as inputs for the FastSpeech2 model during the training phase.

3.3.4 VITS : Direct Character Embedding and Preprocessing

For VITS, once the dataset and the CSV file containing the audio and corresponding transcripts had been prepared, symbols were defined for the entire Nepali language so that each character could receive its own embedding. The characters are then represented as follows:

Cleaned Text: नमस्कार

Character sequence: न, म, स, ण, क, ा, र

Integer sequence: [58, 63, 70, 99, 76, 85, 81]

After sequences were identified, the existing csv for data was taken to create train|val split for the data, (0.9|0.1) The split were represented by filename as first column to help map the audio with the text

```
train/0004.wav|साँझको निलो आकाशमा ताराहरू झिलिमिलि उभिएका छन्, हावा चिसो र सुगन्धित छ, र म आफ्नो झ्यालको छेउमा बसेर यी शान्त क्षणहरूलाई आत्मसात गर्दै छु।  
train/0005.wav|छलफलमा निर्माण व्यवसायी, कन्सल्टयान्ट लगायतका अन्य सम्बद्ध पक्षहरूको पनि सहभागिता रहेको थियो।  
train/0006.wav|नेपाल र चीनको ऐतिहासिक सम्बन्धबारे जोनको धारणा रोचक छ।  
train/0007.wav|सुदूरपश्चिम प्रदेशका केही स्थानमा हल्कादेखि मध्यम वर्षा हुनेछ।  
train/0008.wav|जोन हेल्टन नेपालका एक प्रसिद्ध इतिहासकार हुन्।
```

Figure 3.5: Train | Val split representation for preprocessing

Preprocessing for the VITS model includes generating mel-spectrogram files from the audio, which are saved as ‘.spec.pt’ files. These files are used by Monotonic Alignment Search (MAS) to estimate the duration of each character. This process serves as the VITS equivalent of the Montreal Forced Aligner (MFA) used in the FastSpeech2 model. The model then uses these alignments to learn the relationship between the text and the latent acoustic features, such as pitch and tone, encoded within the spectrogram, allowing the Stochastic Duration Predictor to generate natural speech rhythms.

3.3.5 SpeechT5 : Transliteration and Preprocessing

The main difference in SpeechT5 lies in the preprocessing stage, which begins after the text has been sanitized, cleaned, and a proper dictionary has been created.

The first step is tokenization. As in VITS, integer tokens are assigned to all symbols, and each text sentence is converted into its corresponding token sequence. The preprocessing pipeline then transliterates Nepali text into its Romanized equivalent.

The Romanized text IDs, together with mel labels and speaker embeddings, are padded to equal length within each batch, and masking is applied to train the decoder autoregressively.

Training for SpeechT5

The training process consisted of the following steps:

- Romanized Nepali token IDs are passed into the encoder.
- The decoder autoregressively predicts mel frames, conditioned on the encoder output and speaker embedding.
- The predicted mel-spectrogram is compared with the ground-truth Nepali mel-spectrogram.
- The loss is backpropagated, and the model weights are updated.

Table 3.5: Reperesentation of Nepali in its corresponding English transliteration

Nepali text	English transliteration
काठमाडौँमा अधिकांश समय बादल लागेको छ।	kaatxhamaadxaunmaa adhikaansha samaya baadala laageko chha.</s>

3.3.6 Training Iteration and Refinement

After preprocessing was completed, the individual models were trained using the prepared dataset. During the course of the project, multiple Nepali speech datasets were evaluated. The entire preprocessing pipeline including text normalization, forced alignment, feature extraction, and model training was repeated for each dataset to ensure fair and consistent comparison. Several publicly available Nepali speech datasets were initially considered; however, some were discarded due to issues such as graphemes mapping to noise tokens, inconsistent recordings, and overall poor audio quality.

After evaluation, the IndicTTS Nepali dataset was selected for extended experimentation in addition to the SLR43 dataset, owing to its larger size of approximately 10.32 hours of speech and better coverage of phonetic variation.

FastSpeech2 Training

FastSpeech2 was trained from scratch using the preprocessed Nepali dataset. The training pipeline followed the standard FastSpeech2 procedure: MFA-aligned TextGrid files were used to extract grapheme/phoneme-level duration targets, pitch and energy values were extracted per frame using the preprocessed mel-spectrograms, and all features were normalized before being passed to the model. The model was trained using the Adam optimizer with a warmup learning rate schedule, with multiple runs conducted across varying batch sizes, learning rates, and training step counts to identify stable convergence behaviour. During experimentation, finetuning was also done on the Fastspeech2 model with LJSpeech dataset checkpoint at 900K iterations to test if the model improved.

VITS Fine-Tuning Pipeline

VITS was fine-tuned starting from a pre-trained checkpoint trained on the LJSpeech dataset [6]. The fine-tuning pipeline required several Nepali-specific adaptations. The phoneme set was extended to include Nepali phonemes derived from the custom pronunciation dictionary, and the phoneme embedding layer of the pre-trained model was updated accordingly to accommodate the expanded vocabulary.

Unlike FastSpeech2, VITS does not require external forced alignment as a preprocessing step. Instead, it learns its own internal alignment during training through a monotonic alignment search (MAS) mechanism, which significantly reduced the preprocessing overhead for this model. The model was fine-tuned using the AdamW optimizer with a cosine learning

rate schedule. Due to the adversarial nature of VITS training which jointly optimizes a generator and a multi-period discriminator, a more careful tuning of the learning rate and batch size was required to stabilize training and prevent mode collapse. Multiple fine-tuning configurations were tested across both the SLR43 and IndicTTS datasets, with training runs extended until perceptual output quality stabilized as assessed through informal listening evaluation.

SpeechT5 Fine-Tuning Pipeline

SpeechT5 was fine-tuned from Microsoft’s publicly available `microsoft/speecht5_tts` checkpoint using the Hugging Face Transformers library. The fine-tuning pipeline was comparatively straightforward due to the availability of a well-documented training interface and a pre-trained tokenizer that was extended to include Nepali Devanagari characters, ensuring full script coverage without reliance on transliteration. The model was fine-tuned using the AdamW optimizer with a linear warmup and decay schedule. The fine-tuned model was subsequently paired with a HiFi-GAN vocoder [23] for final waveform synthesis.

This iterative training process across all three architectures allowed systematic comparison of different training strategies from scratch, end-to-end fine-tuning, and pre-trained adaptation all while maintaining a consistent evaluation methodology across the Nepali TTS pipeline.

3.3.7 Testing and Evaluation

The performance of the proposed text-to-speech model is evaluated using a subjective evaluation metric. **Mean Opinion Score (MOS)** evaluates the perceptual quality of synthesized speech through human listening tests. Listeners rate the naturalness and clarity of the generated speech on a scale from 1 (poor) to 5 (excellent).

4. Experimentation & Results

4.1 FastSpeech2 Model Training - Cycle 1 (SLR43 Dataset)

The cleaned, aligned, and preprocessed dataset was used to train the FastSpeech2 neural text-to-speech model directly on Devanagari graphemes, without converting the text into phonemes. This approach ensured that each character, matra, and halanta was explicitly represented in the training data, allowing the model to learn character-level pronunciation patterns for Nepali.

Table 4.1: Sample Nepali Sentences and Corresponding Grapheme Decomposition

Nepali Sentence	Grapheme Representation
आज मौसम राम्रो छ।	आ ज म ौ स म र ा म ्र ो छ ।
हामी बिहान चिया पिउँछौं।	ह ा म ी ब ि ह ा न च ि य ा प ि उँ छ ौ ं ।
कृपया ढोका बन्द गर्नुहोस्।	क रृ प य ा ढ ो क ा ब न् द ग र् नु ह ो स् ।

The dataset consisted of carefully aligned audio-text pairs, where Montreal Forced Aligner (MFA) outputs were used to synchronize speech frames with corresponding graphemes. All preprocessing steps, including normalization of punctuation, removal of noise, and grapheme decomposition, were applied to ensure consistent input sequences. During this first cycle, the model was trained with default hyperparameters and the parameters that showed good result in the literature review.

4.1.1 Acoustic and Model Configuration

Table 4.2: FastSpeech2 Model Configuration

Parameter	Value
Sampling Rate	22050 Hz
FFT Size (n_fft)	1024
Hop Length	256
Mel Channels	80
Mel Frequency Range	0–8000 Hz
Embedding Dimension	256
Hidden Dimension	256
Feed Forward Dimension	1024
Attention Heads	4
Encoder/Decoder Layers	4
Duration , Pitch & Energy Predictor Dimension	256

4.1.2 Training Setup

Training was performed using Google Colab’s Free Tier GPUs. Due to runtime limitations and session resets, check-pointing strategies were adjusted during training. The total training was carried out for 180,000 steps over the time of a week.

Table 4.3: Training Hyperparameters

Parameter	Value
Learning Rate	1×10^{-3}
Warmup Steps	4000
Batch Size	8
Total Training Steps	180,000
Checkpoint Frequency	200 (early) / 4000 (later)

Early results indicated that while the model was able to generate speech that was generally intelligible, some issues with prosody and intonation remained, highlighting the need for larger datasets and further hyperparameter tuning in subsequent cycles. For duration, pitch, and energy predictors, and the mel spectrogram reconstruction loss was closely monitored. The objective of Cycle 1 was to establish a baseline model capable of synthesizing intelligible Nepali speech directly from grapheme sequences, providing a foundation for later experiments with phoneme-based mapping and cluster-based finetuning experimentation.

Training logs were exported in CSV format and visualized using TensorBoard to monitor convergence behavior and detect potential overfitting.

Table 4.4: FastSpeech2 Loss Values of Cycle-1

Metric	Value
Mel Spectrogram Loss	0.3421
Duration Prediction Loss	0.0043
Pitch Prediction Loss	0.0133
Energy Prediction Loss	0.0069
Total Training Loss	0.7062

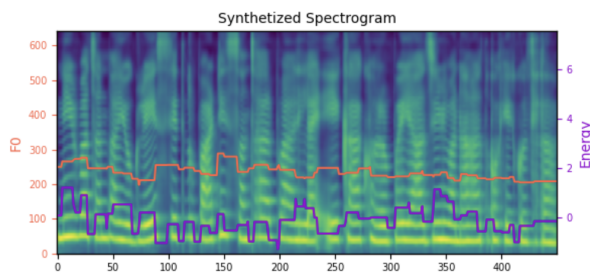


Figure 4.1: MelSpectrogram for Sample Text

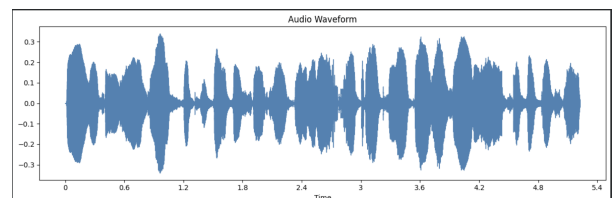


Figure 4.2: Audio Waveform of Generated Audio

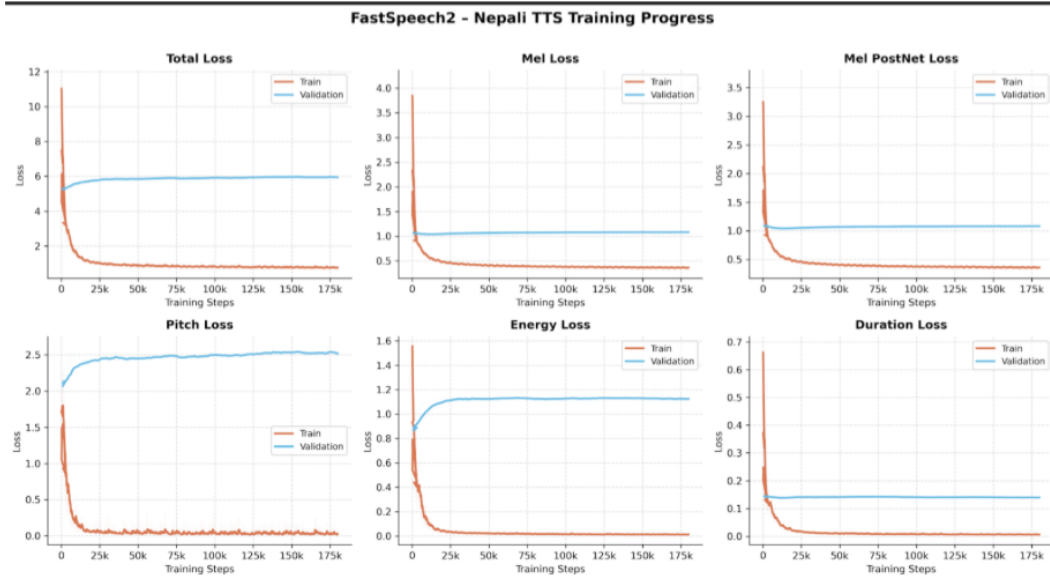


Figure 4.3: Figure: Loss Graphs for run 1

4.2 FastSpeech2 Model Training - Cycle 2 (SLR43 Dataset)

In this iteration, the FastSpeech2 model was trained on the SLR43 dataset using English-based IPA phoneme rules. Each Nepali word in the dataset was converted to its corresponding phoneme sequence, enabling the model to learn pronunciation patterns at a phonetic level rather than relying solely on graphemes. This approach was intended to improve the model’s ability to generate natural-sounding speech by capturing subtle nuances of articulation, stress, and vowel length.

Table 4.5: Sample Nepali Sentences and Phoneme Representations

Nepali Sentence	Phoneme Representation
रेडियो नेपालमा कार्यरत रहेँदा मास्टर रत्नदास प्रकाश निकै लोकप्रिय थिए।	re:ɖio nepa:lama ka:rjə ...
प्रकाश श्रेष्ठ नेपाली गायक हुन्।	praka:s ʃ ref t ^h a: nepa:li ga:jak hun

Table 4.6: FastSpeech2 Loss Metrics of Cycle-2

Metric	Value
Mel Spectrogram Loss	0.4896
Duration Prediction Loss	0.0230
Pitch Prediction Loss	0.0266
Energy Prediction Loss	0.0239
Total Training Loss	1.0464

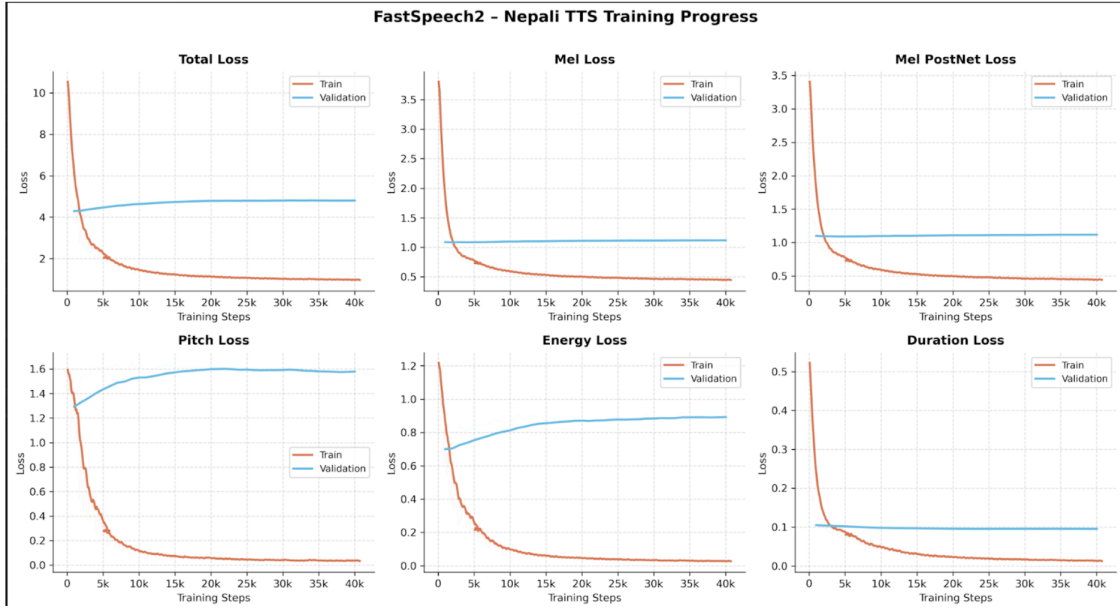


Figure 4.4: Figure: Loss Graph for the phoneme based run

Despite using phoneme-based mapping, the model did not show any significant improvement over the previous grapheme-based training. After 40k training steps, the total training loss was higher than that of the grapheme-based iteration, indicating that the phoneme conversion did not provide a measurable benefit for this dataset. As a result, the training cycle was terminated early, and the insights gained from this experiment were used to guide subsequent grapheme-based finetuning strategies.

Furthermore, as the model had reached a plateau of improvement using only the SLR43 dataset (2.8 hours, 2,064 utterances), efforts were shifted toward exploring larger datasets to improve the quality and naturalness of synthesized speech. This motivated the move to subsequent experiments with IndicTTS and other larger corpora for more robust training and better generalization.

4.3 FastSpeech2 Model Training - Cycle 3 (IndicTTS Dataset)

In Cycle 3, the FastSpeech2 model was trained on the IndicTTS dataset with grapheme based aligning to develop a Nepali text-to-speech system. As the data size was significantly larger than previously used data, multiple training parameters were varied during this cycle, including convolution kernel sizes, number of quantization bins, and weight decay settings, to evaluate their effect on model convergence and audio quality. The experiments focused on grapheme-based input representations to ensure precise character-level pronunciation learning.

Table 4.7: FastSpeech2 Training – Cycle 3 (IndicTTS Dataset)

Category	Parameter / Metric	Value / Observation
Configuration	Weight Decay	0.1 (strong decay)
	Warm-up Step	4000
	Conv Kernel	[3,1] (changed)
	n_bins	64 (changed)
Attention	Query weights std	0.000000 (dead)
	Key weights std	0.000000 (dead)
Validation	Validation Loss	Not meaningful
	Audio Output	Pure flat noise
	Result	FAIL: Weight decay killed attention weights
Training Metrics	Mel Spectrogram Loss	0.5778
	Duration Prediction Loss	0.0358
	Pitch Prediction Loss	0.0462
	Energy Prediction Loss	0.0506
	Total Training Loss	1.2868

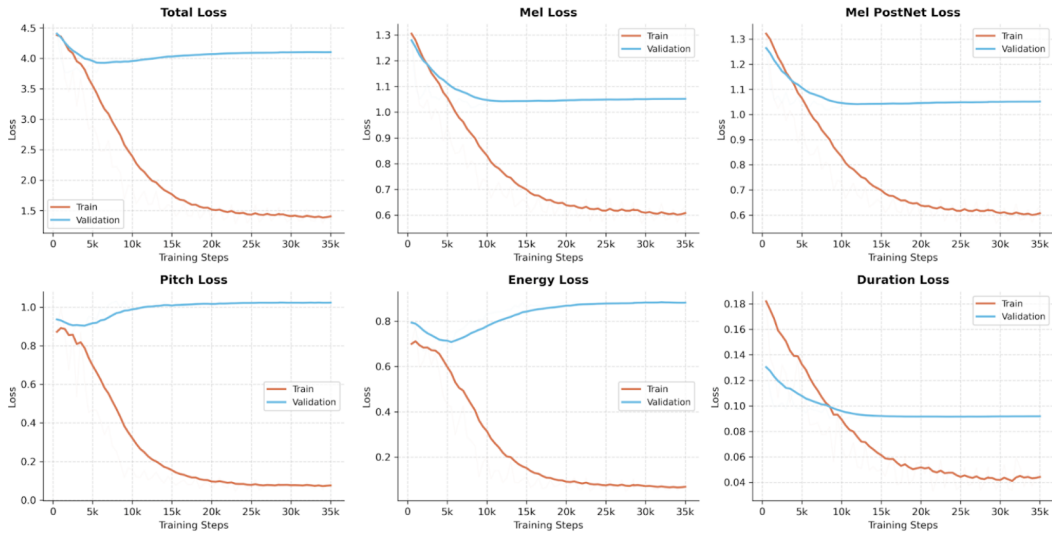


Figure 4.5: Figure: Loss Graph obtained in run 3 changing parameters

4.4 FastSpeech2 Model Finetuning - Cycle 4 (IndicTTS Dataset)

Cluster-based finetuning was performed to evaluate whether the Montreal Forced Aligner (MFA) would perform better on consonant clusters and whether the aligner was correctly handling halanta (ँ) and matras (ः, ऌ, ॑, ॒, ॒, ॒, ॒, ॒, ॒) instead of treating them as empty characters.

Additionally, we observed that previous training cycles were exhibiting severe overfitting, so efforts were made to mitigate overfitting and reduce validation losses. Training parameters were adjusted to decay learning rates sooner and more frequently, with mild weight decay applied to stabilize the updates.

Table 4.8: FastSpeech2 Loss Metrics of Training Cycle-4

Metric	Value
Mel Spectrogram Loss	1.5749
Duration Prediction Loss	0.2704
Pitch Prediction Loss	0.6652
Energy Prediction Loss	0.6782
Total Training Loss	4.7606

Despite these measures, the model continued to exhibit high training losses and generated poor-quality synthesized speech. Consequently, subsequent experiments focused on grapheme-based mapping, which provided more consistent and reliable results for Nepali TTS.

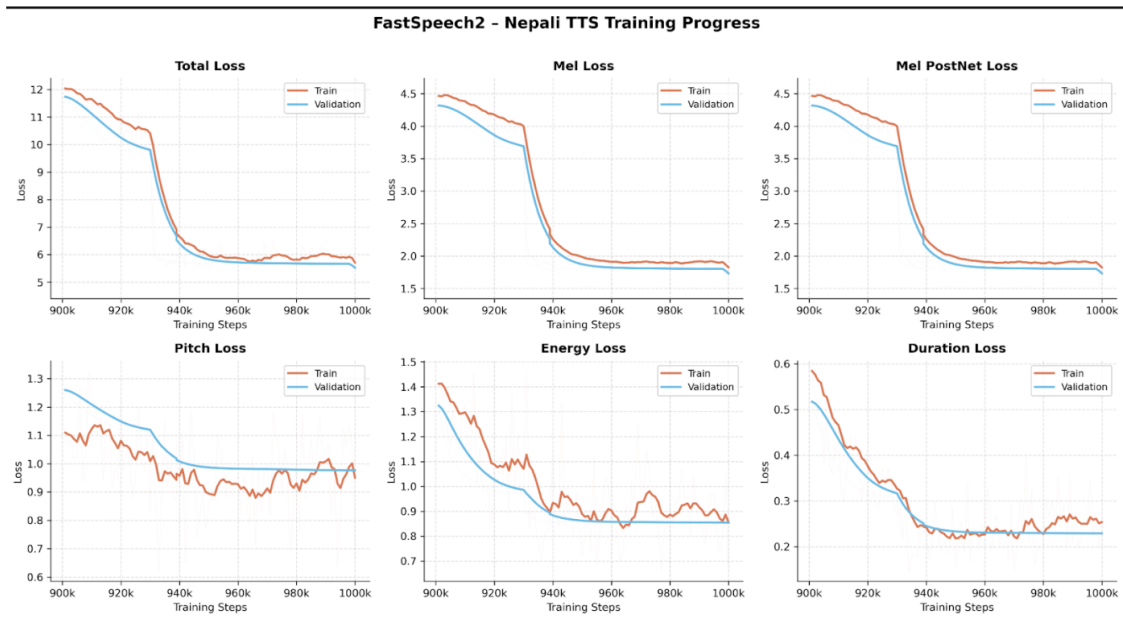


Figure 4.6: Figure: Loss Graph obtained in run 4 changing parameters

4.5 FastSpeech2 Model Finetuning - Cycle 5 (Finetuning on LJSpeech Model with SLR43 Dataset)

In this cycle, the FastSpeech2 TTS model was finetuned using a pre-trained LJSpeech model as the base. The pretraining checkpoint at 900k steps served as the starting point for finetuning.

Table 4.9: FastSpeech2 Loss Metrics of Training Cycle-5

Metric	Value
Mel Spectrogram Loss	0.4585
Duration Prediction Loss	0.0169
Pitch Prediction Loss	0.0332
Energy Prediction Loss	0.0271
Total Training Loss	0.9873

Table 4.10: FastSpeech2 Training Parameters (Optimizer and Step Settings)

Parameter	Value
Optimizer Settings	
Batch Size	12
Betas	[0.9, 0.98]
Epsilon (eps)	1e-9
Weight Decay	0.00001
Gradient Clip Threshold	1.0
Gradient Accumulation Step	1
Warm-up Steps	4000
Anneal Steps	[930000, 960000, 980000]
Anneal Rate	0.3
Total Steps	1,000,000

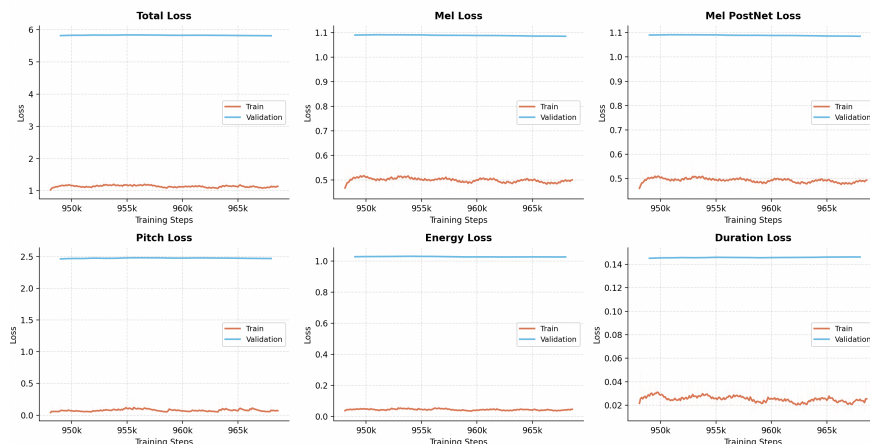


Figure 4.7: Figure: Loss Graph obtained in run 5 with Transfer Learning

4.6 VITS Fine-Tuning on an LJSpeech Checkpoint with the IndicTTS Dataset

4.6.1 Fine-tuning Setup and Configuration

Fine-tuning was initialised from an LJSpeech-pretrained VITS checkpoint. The text encoder vocabulary was replaced with a Nepali-compatible phoneme set, while the decoder and discriminator weights were retained and fine-tuned. The generator and discriminator were optimised simultaneously using separate AdamW optimisers, as is standard in adversarial TTS training. The complete set of hyperparameters is given in Table 4.11.

Table 4.11: VITS Fine-tuning Training Parameters

Parameter	Value	Description
<i>Data and Checkpoints</i>		
Base Checkpoint	pretrained ljspeech (800k steps)	From VITS official github repository [3]
Dataset Used	IndiTTS Nepali Dataset	Monotonic female voice (4950 utterances)
<i>Optimizer Settings (AdamW)</i>		
Learning Rate (Generator)	8×10^{-5}	Generator network learning rate
Learning Rate (Discriminator)	4×10^{-5}	Discriminator network learning rate
Betas	[0.8, 0.99]	Adam momentum coefficients (β_1, β_2)
Epsilon	1×10^{-9}	Numerical stability constant
LR Decay (γ)	0.999875	Exponential decay factor per epoch
<i>Training Schedule</i>		
Batch Size	8	Audio segments per training step
Segment Size	8192	Waveform samples per training segment
Epochs	10000	Maximum training epochs
Gradient Clip Threshold	50.0	Per-element gradient value clip bound
<i>Loss Weights</i>		
λ_{mel} (c_mel)	45	Mel-spectrogram reconstruction loss weight
λ_{kl} (c_kl)	1.0	KL divergence loss weight
λ_{fm} (c_fm)	2.0	Feature matching loss weight
<i>Precision & Logging</i>		
Mixed Precision (FP16)	True	Half-precision training enabled
Log Interval	1000 steps	Steps between training log outputs
Eval Interval	5000 steps	Steps between validation evaluations

The high mel-spectrogram loss weight ($\lambda_{\text{mel}} = 45$) reflects the dominant role of reconstruction fidelity in the early training phase, grounding the generator’s output in the target acoustic space before adversarial shaping becomes the primary driver of audio quality. The relatively low KL weight ($\lambda_{\text{kl}} = 1.0$) allows the posterior encoder some freedom during fine-tuning, preventing posterior collapse while still encouraging the learned latent distribution

to remain close to the prior. The feature matching weight ($\lambda_{\text{fm}} = 2.0$) provides a perceptual shaping signal from the discriminator’s intermediate representations without destabilising the adversarial balance.

Note: Several parameters present in the FastSpeech2 optimizer configuration are absent or structurally different in VITS, owing to fundamental differences in architecture and training paradigm.

- **Weight Decay:** FastSpeech 2 applies weight decay of 1×10^{-5} for regularisation. VITS omits this parameter entirely (defaults to zero), as adversarial discriminator training provides an implicit regularisation effect.
- **Gradient Clipping:** FastSpeech 2 uses norm-based clipping (`clip_grad_norm_`, threshold 1.0), which rescales the full gradient vector preserving its direction. VITS uses value-based clipping (`clip_grad_value_`), clamping each gradient element independently to $[-50.0, +50.0]$. The two methods are not directly comparable.
- **Gradient Accumulation:** FastSpeech 2 supports configurable accumulation steps (set to 1, i.e. no accumulation). VITS does not implement this feature, making the two systems functionally equivalent at a step value of 1.
- **Learning Rate Schedule:** FastSpeech 2 uses a Noam scheduler with 4000 warmup steps, inverse-square-root decay, and discrete annealing at steps 930k–980k. VITS uses `ExponentialLR` with $\gamma = 0.999875$ per epoch and no warmup, which is better suited to its adversarial training loop.
- **Dual Learning Rates:** VITS trains a generator and discriminator simultaneously, requiring separate learning rates (8×10^{-5} and 4×10^{-5}). FastSpeech 2 has no equivalent, as it trains a single model.

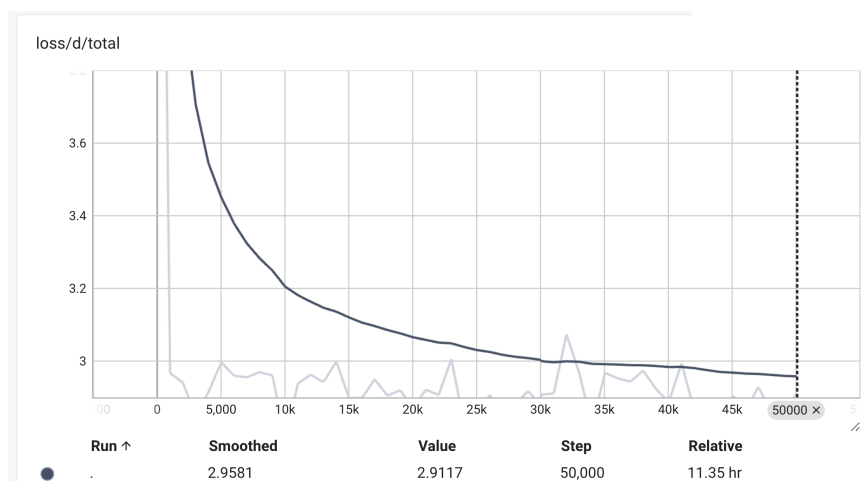


Figure 4.8: Figure: Loss Graph for VITS over 50k steps of training

The mel-spectrogram reconstruction loss ($\lambda_{\text{mel}} = 45$) dominated the early training signal, providing stable gradients before the adversarial components matured. The KL divergence term remained low throughout, indicating that the posterior encoder did not collapse to the prior, and that the Monotonic Alignment Search successfully learned text-to-latent alignments without a pre-trained external aligner.

4.6.2 Results and Audio Quality

The fine-tuned VITS model produced high-fidelity, natural-sounding Nepali speech after the full training run. Key observations from the synthesised output include:

- **End-to-end waveform synthesis:** Unlike pipeline systems, VITS directly produces raw audio without a separate vocoder stage, eliminating the spectral smoothing artefacts common in two-stage systems.
- **Natural prosody:** The stochastic duration predictor generated varied, natural-sounding rhythm and timing across utterances, avoiding the monotone delivery that can affect deterministic duration models.
- **Phoneme coverage:** Nepali retroflex and aspirated consonants, which are absent from the LJSpeech English pre-training data, were correctly rendered following fine-tuning on the IndicTTS corpus.
- **Audio fidelity:** The HiFi-GAN decoder, retained from the LJSpeech checkpoint and fine-tuned, produced clean, high-resolution waveforms with minimal background noise.

4.7 SpeechT5 Finetuning with SLR43

4.7.1 Fine-tuning Setup and Configuration

Fine-tuning was performed starting from the official `microsoft/speecht5_tts` checkpoint using the Hugging Face `Seq2SeqTrainer` API, with a custom `data_collator` handling dynamic padding of mel-spectrogram frames and text token sequences within each batch. Transfer learning from this checkpoint significantly reduced the data and compute required to achieve good-quality Nepali speech synthesis compared to training from scratch. The complete set of training hyperparameters is presented in Table 4.12

An effective batch size of 32 was achieved by combining a per-device batch size of 4 with 8 gradient accumulation steps, enabling stable gradient estimation without requiring a multi-GPU setup. A learning rate of 1×10^{-4} with a 100-step linear warmup prevented large gradient updates during the early phase of fine-tuning, when the model’s weights were still adapting from English to Nepali acoustic patterns. Mixed-precision training (`fp16=True`) and gradient checkpointing were both enabled to reduce GPU memory consumption, making fine-tuning feasible on a single consumer-grade GPU. Evaluation was performed every 100

Table 4.12: SpeechT5 fine-tuning hyperparameters on SLR43

Parameter	Value	Description
<i>Data and checkpoint</i>		
Base checkpoint	Microsoft’s Chkpt	Pre-trained TTS model from
Training dataset	SLR43 train split	Hugging Face
Evaluation dataset	SLR43 test split	Nepali read-speech corpus
		Held-out evaluation utterances
<i>Batch and optimisation</i>		
Batch size (train)	4	Per-device training batch size
Batch size (eval)	2	Per-device evaluation batch
Gradient accumulation	8	size Effective batch size = $4 \times 8 =$
Learning rate	1×10^{-4}	32 AdamW optimiser learning
Warmup steps	100	rate
Max training steps	2,500	Linear LR warmup from zero
		Total number of optimisation
		steps
<i>Logging and checkpointing</i>		
Evaluation interval	Every 100 steps	Periodic validation during
Logging interval	Every 200 steps	training
Checkpoint save	Every 5,000 steps	Loss logged to console
		Model weights saved to output
		directory
<i>Memory and precision</i>		
Mixed precision	Enabled (fp16)	Half-precision for speed and
Gradient checkpointing	Enabled	memory reduction
Best model loading	Disabled	Trades recomputation for re-
		duced GPU memory
		Final checkpoint used (not
		best validation loss)

steps on the held-out SLR43 test split, with only the scalar prediction loss reported — sufficient for monitoring convergence without the overhead of full spectrogram decoding at each checkpoint.

The model was trained end-to-end with a combined objective consisting of a mel-spectrogram reconstruction loss ($\mathcal{L}_1$) and a duration/attention auxiliary loss, which encouraged monotonic alignment between input text tokens and output acoustic frames.

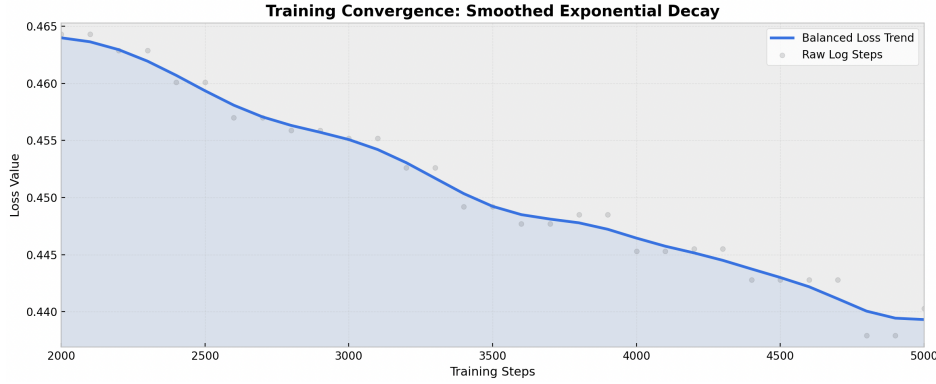


Figure 4.9: Figure: Loss Graph for SpeechT5 over 5000 steps of training

Figure 4.9 presents the training loss curve recorded over approximately 3,500 steps. As shown, the loss exhibits a characteristic rapid descent during the early phase (steps 0–500), reflecting the model’s quick adaptation of pre-trained English representations toward Nepali phonetics. Beyond step 500, the loss continues to decrease at a more gradual and stable rate, indicating steady convergence without evidence of overfitting or divergence.

The smooth and consistent decline in training loss validates that the fine-tuning procedure was stable, and that the SLR43 corpus provided sufficient acoustic signal for the model to adapt its pre-trained English representations to Nepali speech.

4.7.2 Results and Audio Quality

After 5,000 training steps (approximately 4.5 hours of wall-clock time), the fine-tuned SpeechT5 model produced intelligible and natural-sounding Nepali speech. The synthesised audio demonstrated the following characteristics:

- **Correct Nepali phoneme rendering:** The model accurately vocalised Devanagari-mapped phonemes absent from English, including retroflex and aspirated consonants.
- **Natural prosody and rhythm:** Fluent intonation patterns were produced, attributable to the attention-based alignment mechanism inherited from the pre-trained backbone.
- **Consistent speaker identity:** The model conditions on x-vector speaker embeddings, which maintained a stable voice identity across synthesised utterances.

The HiFi-GAN vocoder was critical as a post-processing stage, converting predicted mel-spectrograms into high-fidelity raw audio waveforms and avoiding the muffled quality associated with traditional Griffin-Lim reconstruction.

5. Results and Comparative Evaluation

Table 5.1: Comparative Overview of FastSpeech2, VITS, and SpeechT5

Feature	FastSpeech2	VITS	SpeechT5
Architecture	Non-autoregressive Feed-Forward Transformer with variance adaptor	End-to-end VAE with normalizing flows and adversarial training	Unified encoder-decoder transformer pre-trained on speech and text
Pipeline type	Two-stage: acoustic model + separate vocoder (HiFi-GAN)	Single-stage: acoustic modelling and vocoding unified	Two-stage: acoustic model + separate vocoder (HiFi-GAN)
Alignment method	External forced aligner required (MFA) for duration targets	Internal monotonic alignment search (MAS) — no external aligner	Attention-based alignment learned during pre-training
Inference speed	Up to 38× faster than autoregressive baselines [2]	Real-time factor comparable to two-stage systems despite end-to-end design	Moderate; pre-trained transformer adds overhead at inference
Prosodic control	Explicit control over pitch, energy, and duration via variance adaptor	Implicit prosody through latent variable sampling; less direct control	Limited explicit prosodic control; relies on pre-trained representations
Speech naturalness	Competitive with autoregressive models; MOS comparable to Tacotron 2 [2]	State-of-the-art naturalness; outperforms two-stage systems on MOS [3]	High naturalness via cross-modal pre-training; strong on seen speakers
Data requirement	Moderate; requires clean paired speech-text data with alignment	Moderate to high; benefits greatly from larger datasets for flow training	Low for fine-tuning; leverages large-scale pre-training on English data
Low-resource suitability	Reasonable; parallel inference and explicit structure help under data constraints	Challenging; end-to-end pre-training is harder to stabilize on limited data	Strong for fine-tuning; pre-trained weights compensate for scarce target data
Speaker adaptation	Single-speaker by default; multi-speaker via speaker embeddings	Native multi-speaker support via speaker conditioning	Built-in speaker embedding for speaker-adaptive synthesis
Vocoder dependency	Requires separate vocoder (HiFi-GAN); quality depends on vocoder	No separate vocoder; waveform generated end-to-end	Requires separate vocoder (HiFi-GAN)
Training complexity	Relatively simple; stable training with clear loss functions	Complex; adversarial training prone to instability	Simple fine-tuning from pre-trained checkpoint; low setup overhead
Key strength	Fast inference, explicit prosodic controllability, training stability	Highest naturalness, no aligner dependency, end-to-end simplicity	Strong transfer learning, cross-modal pre-training, task versatility
Key limitation	Requires external aligner (MFA); quality sensitive to alignment accuracy	Harder to train on limited data; less interpretable prosody control	Less explicit prosodic control; dependent on pre-training language quality
Original language	English and Mandarin [2]	English (LJ Speech) [3]	English [4]

Table 5.2 presents a comparative analysis of Mean Opinion Scores (MOS) across several Nepali text-to-speech (TTS) systems developed in prior studies, alongside the results obtained from our proposed approaches based on FastSpeech2, VITS, and SpeechT5. The evaluation was conducted with 15 human evaluators, each of whom rated audio samples on a five-point Likert scale (1–5) across five perceptual dimensions: naturalness, clarity, prosodic flow, audio quality, and overall impression.

Table 5.2: Comparison of Mean Opinion Scores (MOS) Across Prior Nepali TTS Systems and the Proposed Approaches

Model	MOS Score
<i>Current Approach</i>	
FastSpeech2	2.87
VITS	3.20
SpeechT5	3.86
<i>Past Models</i>	
WaveNet [8]	3.07
FastPitch + HiFi-GAN [13]	3.70
Tacotron 2 + HiFi-GAN [9]	4.04

5.1 Discussion

5.1.1 FastSpeech2

Efforts were initially directed toward identifying training and linguistic strategies that could improve the performance of FastSpeech2 for Nepali. Because a robust Nepali FastSpeech2 pipeline, including a well-documented Montreal Forced Aligner (MFA) dictionary and lexicon, was not readily available, and because detailed architectural and hyperparameter references for Indic-language implementations were limited, the training process relied heavily on systematic experimentation.

Using the default FastSpeech2 hyperparameters from the English and Chinese TTS implementation described by [27] as a baseline, the model began to overfit after approximately 30,000–40,000 steps. In practice, this produced partially usable outputs in which the first few words of a test sentence were intelligible, but the latter half often degraded into metallic or unstable noise.

Subsequent attempts to mitigate overfitting focused on tuning several critical hyperparameters:

- Dropout rate

- Weight decay
- Annealing steps and annealing rate
- Warmup step size

All attempts showed the same pattern: validation loss either spiked or plateaued shortly after the warmup stage, while training loss continued to decrease. This was a clear indication of overfitting. Parallel experiments with alternative data processing strategies, including phonetic conversion and direct grapheme cluster-based inference, were also explored, but they did not yield any better measurable improvement.

As a result, the focus shifted toward VITS and SpeechT5, both of which produced much better perceptual quality on the same dataset. The MOS score of FastSpeech2 remained the lowest among the three models evaluated in this work, at 2.87, confirming that direct training from scratch under the available low-resource conditions was not sufficient to achieve competitive Nepali TTS quality with this architecture.

5.1.2 VITS

Fine-tuning VITS from an LJSpeech checkpoint proved effective for Nepali TTS, demonstrating that the model’s end-to-end latent speech representations transfer reasonably well across languages, particularly when only a limited amount of target-language data is available. The joint training objective, which combines reconstruction, adversarial, feature-matching, and KL-divergence losses, provided complementary supervision signals that helped shape both the acoustic quality and the naturalness of the generated speech.

Compared with SpeechT5, VITS required considerably more optimisation steps to reach a comparable perceptual quality. In this study, VITS required around 50,000 fine-tuning steps, whereas SpeechT5 produced usable output after approximately 5,000 steps. This difference reflects the greater complexity of stabilising adversarial training, especially when adapting from a pre-trained checkpoint to a low-resource target language.

At the same time, VITS offered an important architectural advantage: it integrated acoustic modelling and waveform generation into a single end-to-end framework. This removed the dependency on a separately trained vocoder and reduced the risk of artefacts introduced at the boundary between mel-spectrogram generation and waveform synthesis. Its MOS score of 3.20 placed it above FastSpeech2, showing that end-to-end transfer learning is a more viable direction than training from scratch, though still below the best result achieved by SpeechT5.

5.1.3 SpeechT5

The relatively brief fine-tuning run of 5,000 steps was sufficient to produce usable and perceptually strong Nepali TTS output, underscoring the effectiveness of transfer learning from a

large pre-trained backbone. Starting from `microsoft/speecht5_tts` meant that the model’s internal representations of prosody, duration, and acoustic structure were already mature; fine-tuning primarily redirected these representations toward Nepali phonetics and the acoustic characteristics of the SLR43 speakers.

A major advantage of SpeechT5 over task-specific architectures such as Tacotron 2 or VITS is its unified pre-training objective, which makes the backbone more robust to domain shift and reduces the number of task-specific parameters that must be learned during adaptation. This property made SpeechT5 particularly well-suited to low-resource Nepali TTS fine-tuning, where large in-domain corpora are not readily available.

This advantage was also reflected in the evaluation results. SpeechT5 achieved the highest MOS score among the three models implemented in this work, with a score of 3.86 when surveyed with 15 people. Although this result seems to remain below the 4.04 MOS reported for Tacotron 2 + HiFi-GAN in prior Nepali work [9], it outperformed both WaveNet [8] and FastPitch + HiFi-GAN [13], demonstrating that pre-trained transformer-based transfer learning is a highly promising direction for future Nepali TTS development.

However, a closer examination of SpeechT5’s output reveals a limitation that MOS scores alone do not fully capture. Pre-trained on English speech data, the model’s internal representations of prosody, rhythm, and intonation are heavily influenced by English acoustic properties, and fine-tuning on limited Nepali data is insufficient to fully override these patterns. Nepali has fundamentally different tonal and rhythmic characteristics that the model struggles to adapt to under low-resource conditions.

Unlike FastSpeech2, and VITS, the SpeechT5 model lacks a variance adaptor and stochastic duration predictor, meaning it has no explicit mechanism to introduce natural variation in syllable duration, pitch, or energy. Combined with its deterministic inference behaviour, the model produces the same flat prosodic output for a given input on every pass lacking the natural variability a human speaker would provide. The result is speech that is intelligible but monotonous, feeling unnatural upon closer listening despite its strong MOS score.

Extended fine-tuning on a larger, prosodically diverse Nepali dataset with more training steps would likely yield meaningful gains. At only 5,000 fine-tuning steps, SpeechT5 has clearly not reached its ceiling for Nepali synthesis.

6. Conclusion

Access to high-quality speech synthesis remains largely unavailable for Nepali and many other low-resource languages, creating a significant barrier to digital accessibility for millions of speakers. This project addressed that gap by implementing, experimenting with, and comparatively evaluating three neural TTS architectures : FastSpeech2, VITS, and SpeechT5, for Nepali speech synthesis, alongside an end-to-end pipeline incorporating OCR-based text extraction, normalization, forced alignment, and waveform synthesis.

The central finding of this work is that training strategy matters as much as model architecture under low-resource conditions. FastSpeech2, implemented from scratch, demonstrated the depth of challenges inherent to building native Nepali TTS without pre-trained support, offering useful prosodic controllability in principle, but remaining highly sensitive to data scarcity and alignment quality in practice. VITS, fine-tuned from a pre-trained checkpoint, produced more natural output through its end-to-end generative design, though at significantly higher computational cost. SpeechT5 achieved the strongest results, delivering intelligible Nepali speech after only 5,000 fine-tuning steps and attaining the highest MOS score of 3.86 demonstrating that large pre-trained transformer backbones can effectively compensate for limited in-domain data.

These results carry a clear implication for low-resource TTS research: transfer learning from strong multilingual or cross-lingual pre-trained models currently represents the most practical path to quality Nepali speech synthesis, and native from-scratch training remains constrained by the availability of sufficient, well-curated Nepali speech data. At the same time, MOS scores alone did not capture the full picture as SpeechT5’s outputs were also a bit prosodically monotonous, lacking the natural variation that architectures with explicit prosodic control can in principle provide.

This project establishes both a functional end-to-end Nepali TTS prototype and a documented experimental baseline that future work can build upon whether through larger datasets, stronger pre-trained backbones, or deployment as an accessible web extension for Nepali-speaking communities.

7. Limitations and Future Enhancements

7.1 Limitations

Although a functional Nepali TTS system was successfully developed using FastSpeech2, VITS, and SpeechT5, several limitations were encountered during the development and evaluation phases. The key limitations of this project are outlined below:

- **Data Scarcity and FastSpeech2 Generalization Failure:** The most significant limitation of this work was the scarcity of high-quality Nepali speech data. The available datasets were considerably smaller than those typically used in state-of-the-art English TTS systems, restricting phonetic diversity, vocabulary coverage, and prosodic variation. This had a particularly severe impact on FastSpeech2, which relies heavily on explicit duration prediction and well-aligned training pairs. The model consistently struggled to generalize to unseen or longer utterances, frequently producing broken prosody, incorrect stress patterns, and unintelligible output on out-of-distribution inputs. Despite extensive experimentation with preprocessing pipelines, dictionary construction, and alignment tuning, FastSpeech2 remained the most problematic model throughout the project, highlighting the fundamental mismatch between its architectural assumptions and the constraints of low-resource Nepali data.
- **Limited Nepali Language Resources:** The absence of mature, language-specific tools for Nepali introduced substantial engineering overhead. Reliable grapheme-to-phoneme converters, pronunciation lexicons, and forced-alignment resources for Devanagari-based Nepali remain underdeveloped, necessitating extensive manual experimentation in normalization, dictionary creation, and preprocessing. This increased pipeline complexity and introduced potential error sources at multiple stages of the system.
- **Lack of Nepali-Focused OCR:** The OCR component of the pipeline relies on general-purpose engines not specifically optimized for Nepali Devanagari script. This results in extraction errors on complex conjunct characters, uncommon glyphs, and low-quality image inputs, introducing noise into the text normalization stage before synthesis even begins. A Nepali-native OCR model would significantly improve end-to-end pipeline reliability.
- **High Computational Cost of VITS and Scalability Concerns:** The VITS architecture, while producing more natural-sounding output than FastSpeech2, is signif-

icantly more computationally demanding due to its end-to-end generative design combining a variational autoencoder with a flow-based decoder. Training VITS required substantially more time and GPU resources compared to the other models. More critically, extending VITS to a multi-speaker setting would require considerably higher compute, larger datasets with speaker labels, and longer training schedules. This scalability constraint poses a practical barrier for future improvements within resource-limited environments.

- **Single-Speaker Output:** All three models were trained and evaluated on a single-speaker Nepali dataset. The resulting systems therefore reflect one voice identity and do not generalize to diverse speaking styles, accents, or speaker characteristics. Real-world deployment would benefit from multi-speaker training to improve naturalness and inclusivity across different user demographics.
- **No Mixed-Language or Colloquial Support:** The system is scoped to standard digital Nepali text and does not handle code-switched content, mixed Nepali-English input, colloquial speech patterns, or dialect-specific vocabulary. Such inputs are common in real-world Nepali digital text and would require dedicated handling at the normalization and phoneme conversion stages.
- **Small-Scale Subjective Evaluation:** The MOS evaluation, while providing useful comparative insight, was conducted with only 15 human evaluators and remains inherently subjective. A more rigorous evaluation with a larger listener pool and additional objective acoustic metrics such as Mel Cepstral Distortion (MCD) or Word Error Rate (WER) via ASR would provide stronger and more generalizable evidence of synthesis quality.

7.2 Future Enhancements

Several directions can build upon the foundation established by this work, ranging from dataset expansion and model improvements to real-world application deployment:

- **Web Extension Deployment:** A primary planned extension of this project is the integration of the best-performing TTS pipeline into a publicly accessible browser extension which would allow users to select any Nepali text on a webpage and have it read aloud directly. The extension would build on the existing OCR–TTS pipeline and expose prosodic controls such as speed and pitch adjustment for end-user customization.
- **Real-Time Mobile and Desktop Application:** Packaging the TTS system as a lightweight real-time application for mobile and desktop platforms would significantly expand its accessibility reach. Achieving real-time performance on mobile hardware

would require model compression techniques such as knowledge distillation, quantization, or pruning to reduce inference latency without sacrificing output quality.

- **Bidirectional Speech Interface:** Integrating a speech-to-text component alongside the TTS pipeline would create a complete bidirectional Nepali speech interface enabling both text-to-speech and speech-to-text within a single application. This would enable voice-driven interaction with Nepali digital content, with direct applications in assistive tools, voice search, and hands-free accessibility features.
- **Expand Nepali Speech Datasets:** Curating larger, multi-speaker Nepali speech corpora with broader lexical and prosodic coverage remains the most impactful step toward improving model quality across all three architectures. Community-driven data collection efforts and crowdsourced recording platforms could accelerate this process without requiring institutional resources.
- **Nepali-Native OCR Integration:** Replacing the general-purpose OCR engine with a Nepali-optimized model trained specifically on Devanagari script including conjunct characters and common font variations would reduce extraction errors and improve end-to-end pipeline reliability for real-world document and image inputs.
- **Improved Language-Specific Preprocessing:** Developing Nepali-specific resources including standardized pronunciation dictionaries, grapheme-to-phoneme rules, and more reliable forced-alignment pipelines would directly address the preprocessing bottlenecks encountered in this work and reduce the manual overhead required for future Nepali TTS research.
- **Larger-Scale Evaluation:** Future evaluations should incorporate objective metrics such as Mel Cepstral Distortion (MCD), Word Error Rate (WER) via ASR-based verification, and speaker similarity scores alongside MOS, conducted across larger and more diverse listener pools to produce more statistically robust quality assessments.

Overall, future progress in Nepali TTS will depend on the combination of richer datasets, stronger pre-trained backbones, and language-specific preprocessing infrastructure. The results of this project suggest that such improvements particularly when paired with practical deployment could substantially narrow the quality gap between Nepali TTS and systems developed for high-resource languages, while making synthesized Nepali speech genuinely accessible to the communities that need it most.

References

- [1] Keshan Sodimana, Knot Pipatsrisawat, Linne Ha, Martin Jansche, Oddur Kjartansson, Pasindu De Silva, and Supheakmungkol Sarin. A Step-by-Step Process for Building TTS Voices Using Open Source Data and Framework for Bangla, Javanese, Khmer, Nepali, Sinhala, and Sundanese. In *Proc. The 6th Intl. Workshop on Spoken Language Technologies for Under-Resourced Languages (SLTU)*, pages 66–70, Gurugram, India, August 2018.
- [2] Yi Ren, Chenxu Hu, Xu Tan, Tao Qin, Sheng Zhao, Zhou Zhao, and Tie-Yan Liu. Fast-speech 2: Fast and high-quality end-to-end text to speech. In *International Conference on Learning Representations*, 2021.
- [3] Jaehyeon Kim, Jungil Kong, and Juhee Son. Conditional variational autoencoder with adversarial learning for end-to-end text-to-speech. In *Proceedings of the 38th International Conference on Machine Learning*, 2021.
- [4] Junyi Ao, Rui Wang, Long Zhou, Chengyi Wang, Shuo Ren, Yu Wu, Shujie Liu, Tom Ko, Qing Li, Yu Zhang, Zhihua Wei, Yao Qian, Jinyu Li, and Furu Wei. Specht5: Unified-modal encoder-decoder pre-training for spoken language processing. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5723–5738, Dublin, Ireland, 2022. Association for Computational Linguistics.
- [5] Michael McAuliffe, Michaela Socolof, Sarah Mihuc, Michael Wagner, and Morgan Sonderegger. Montreal forced aligner: Trainable text-speech alignment using kaldi. In *Proc. Interspeech 2017*, pages 498–502, 2017.
- [6] Keith Ito and Linda Johnson. The LJ speech dataset. <https://keithito.com/LJ-Speech-Dataset/>, 2017.
- [7] Roop Bajracharya, Santosh Regmi, Bal Krishna Bal, and Balaram Prasain. Building a natural sounding text-to-speech system for the nepali language: research and development challenges and solutions. *Gipan*, 4:106–116, December 2019.
- [8] Ashok Basnet. Attention and wave net vocoder based nepali text-to-speech synthesis. <https://elibrary.tucl.edu.np/handle/123456789/7668>, 2021.
- [9] Supriya Khadka, Ranju G.C., Prabin Paudel, Rahul Shah, and Basanta Joshi. Nepali text-to-speech synthesis using tacotron2 for melspectrogram generation. In *SIGUL 2023*,

2nd Annual Meeting of the Special Interest Group on Under-resourced Languages: a Satellite Workshop of Interspeech 2023, pages 73–77, August 2023.

- [10] Yi Ren, Yangjun Ruan, Xu Tan, Tao Qin, Sheng Zhao, Zhou Zhao, and Tie-Yan Liu. FastSpeech: Fast, robust and controllable text to speech. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 3171–3180, 2019.
- [11] Alina Lamichhane, Basanta Joshi, and Bibha Sthapit. Nepali text-to-speech using fast-speech and melgan model. In *Proceedings of the 14th IOE Graduate Conference*, volume 14, December 2023. Peer Reviewed.
- [12] G. K. Kumar, P. S. V, P. Kumar, M. M. Khapra, and K. Nandakumar. Towards building text-to-speech systems for the next billion users. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP 2023)*, 2023.
- [13] I. Dongol and B. K. Bal. Transformer-based nepali text-to-speech. In *Proceedings of the 20th International Conference on Natural Language Processing (ICON 2023)*, pages 651–656, Goa, India, December 2023.
- [14] Rupak Raj Ghimire, Bal Krishna Bal, Balaram Prasain, and Prakash Poudyal. Pronunciation-aware syllable tokenizer for nepali automatic speech recognition system. In *Proceedings of the 20th International Conference on Natural Language Processing (ICON)*, pages 36–43, Goa University, Goa, India, December 2023. NLP Association of India (NLP AI).
- [15] Aaron van den Oord, Sander Dieleman, Heiga Zen, Karen Simonyan, Oriol Vinyals, Alex Graves, Nal Kalchbrenner, Andrew Senior, and Koray Kavukcuoglu. Wavenet: A generative model for raw audio. In *Proceedings of the 9th ISCA Speech Synthesis Workshop*, 2016.
- [16] Yuxuan Wang, R.J. Skerry-Ryan, Daisy Stanton, Yonghui Wu, Ron J. Weiss, Navdeep Jaitly, Zongheng Yang, Ying Xiao, Zhifeng Chen, Samy Bengio, Quoc Le, Yannis Agiomyrgiannakis, Rob Clark, and Rif A. Saurous. Tacotron: Towards end-to-end speech synthesis. In *Proceedings of Interspeech*, pages 4006–4010, 2017.
- [17] Jonathan Shen, Ruoming Pang, Ron J. Weiss, Mike Schuster, Navdeep Jaitly, Zongheng Yang, Zhifeng Chen, Yu Zhang, Yuxuan Wang, R.J. Skerry-Ryan, Rif A. Saurous, Yannis Agiomyrgiannakis, and Yonghui Wu. Natural TTS synthesis by conditioning WaveNet on mel spectrogram predictions. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 4779–4783, 2018.

- [18] Kai Shen, Zeqian Ju, Xu Tan, Yanqing Liu, Yichong Leng, Lei He, Tao Qin, Sheng Zhao, and Jiang Bian. NaturalSpeech2: Latent diffusion models are natural and zero-shot speech and singing synthesizers. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2024.
- [19] Yinghao Aaron Li, Cong Han, Vinay S. Raghavan, Gauthier Mischler, and Nima Mesgarani. StyleTTS2: Towards human-level text-to-speech through style diffusion and adversarial training with large speech language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [20] Matthew Le, Apoorv Vyas, Bowen Shi, Brian Karrer, Leda Sari, Rashel Moritz, Mary Williamson, Vimal Manohar, Yossi Adi, Jay Mahadeokar, and Wei-Ning Hsu. Voicebox: Text-guided multilingual universal speech generation at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [21] Chengyi Wang, Sanyuan Chen, Yu Wu, Ziqiang Zhang, Long Zhou, Shujie Liu, Zhuo Chen, Yanqing Liu, Huaming Wang, Jinyu Li, Lei He, Sheng Zhao, and Furu Wei. VALL-E: Neural codec language models are zero-shot text to speech synthesizers. In *arXiv preprint arXiv:2301.02111*, 2023.
- [22] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research*, 21(140):1–67, 2020.
- [23] Jungil Kong, Jaehyeon Kim, and Jaekyoung Bae. Hifi-gan: Generative adversarial networks for efficient and high fidelity speech synthesis. In *Advances in Neural Information Processing Systems*, volume 33, 2020.
- [24] Ethnologue. Nepali. <https://www.ethnologue.com/language/npi/>, 2023.
- [25] Bhuvana Narasimhan, Richard Sproat, and George Kiraz. A proposed experiment in hindi text-to-speech synthesis. In *Workshop on TEXT, SPEECH AND DIALOGUE*, 1998.
- [26] Monojit Choudhury, Anupam Basu, and Sudeshna Sarkar. Rule-based schwa deletion in hindi and nepali. In *International Symposium on Computational Linguistics*, 2004.
- [27] Chung-Ming Chien, Jheng-Hao Lin, Chien-yu Huang, Po-chun Hsu, and Hung-yi Lee. Investigating on incorporating pretrained and learnable speaker representations for multi-speaker multi-style text-to-speech. In *ICASSP 2021 - 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 8588–8592, 2021.